

---

# **Quantitative Big Imaging - Complex shapes**

**Anders Kaestner**

**Mar 29, 2026**



## CONTENTS

|      |                                                       |    |
|------|-------------------------------------------------------|----|
| 0.1  | Complex Objects and Distributions . . . . .           | 1  |
| 0.2  | Structure analysis . . . . .                          | 6  |
| 0.3  | Skeletonization / Networks . . . . .                  | 7  |
| 0.4  | Network analysis explained in real images . . . . .   | 8  |
| 0.5  | Skeletonization - take one . . . . .                  | 13 |
| 0.6  | Skeletonization with Morphological thinning . . . . . | 17 |
| 0.7  | Skeleton: Junction Overview . . . . .                 | 18 |
| 0.8  | Dedicated pruning algorithms . . . . .                | 21 |
| 0.9  | Establish Topology . . . . .                          | 25 |
| 0.10 | Skeleton analysis - Tortuosity . . . . .              | 32 |
| 0.11 | Segmenting touching items . . . . .                   | 37 |
| 0.12 | Watershed: Flowing Downhill . . . . .                 | 40 |
| 0.13 | Color maps for many labels . . . . .                  | 49 |
| 0.14 | Example - Separating grain packings . . . . .         | 50 |
| 0.15 | Summary . . . . .                                     | 63 |
| 0.16 | Next time on QBI . . . . .                            | 64 |



This is the lecture notes for the 6th lecture of the Quantitative big imaging class given during the spring semester 2021 at ETH Zurich, Switzerland.

## 0.1 Complex Objects and Distributions

### 0.1.1 Literature / Useful References

#### Books

- Jean Claude, Morphometry with R
- John C. Russ, The Image Processing Handbook, (Boca Raton, CRC Press)
- Pierre Soille,

#### Papers / Sites

##### Voronoi Tessellations

- Ghosh, S. (1997). Tessellation-based computational methods for the characterization and analysis of heterogeneous microstructures. Composites Science and Technology, 57(9-10), 1187–1210
- Wolfram Explanation

##### Self-Avoiding / Nearest Neighbor

- Schwarz, H., & Exner, H. E. (1983). The characterization of the arrangement of feature centroids in planes and volumes. Journal of Microscopy, 129(2), 155–169.
- Kubitscheck, U. et al. (1996). Single nuclear pores visualized by confocal microscopy and image processing. Biophysical Journal, 70(5), 2067–77.

##### Alignment / Distribution Tensor

- Mader, K. et al (2013). A quantitative framework for the 3D characterization of the osteocyte lacunar system. Bone, 57(1), 142–154
- Aubouy, M., et al. (2003). A texture tensor to quantify deformations. Granular Matter, 5, 67–70.

##### Two point correlation

- Dinis, L., et. al. (2007). Analysis of 3D solids using the natural neighbour radial point interpolation method. Computer Methods in Applied Mechanics and Engineering, 196(13-16)

```
import seaborn as sns
from skimage.morphology import skeletonize
from skimage.morphology import binary_opening, binary_closing, disk
from skimage.morphology import grayreconstruct as gr
from scipy.ndimage import distance_transform_edt
import numpy as np
from skimage.color import hsv2rgb, rgb2hsv
from skimage.morphology import medial_axis
from skimage.filters import laplace
from skimage.segmentation import mark_boundaries
import seaborn as sns
from skimage.morphology import label
from scipy.ndimage import convolve
```

(continues on next page)

(continued from previous page)

```
from matplotlib.patches import Rectangle
from matplotlib.patches import ConnectionPatch

from skimage.morphology import opening, closing, disk # for removing small objects
import matplotlib.pyplot as plt # for showing plots
from skimage.io import imread # for reading images
import pandas as pd # for reading the swc files (tables of somesort)
import sys
sys.path.append('../common/')
import plotsupport as ps

import warnings
warnings.simplefilter(action='ignore', category=FutureWarning)

%matplotlib inline

from matplotlib.colors import ListedColormap
```

### 0.1.2 Outline

- Motivation (Why and How?)
- Skeletons
- Tortuosity
- Watershed Segmentation
- Connected Objects

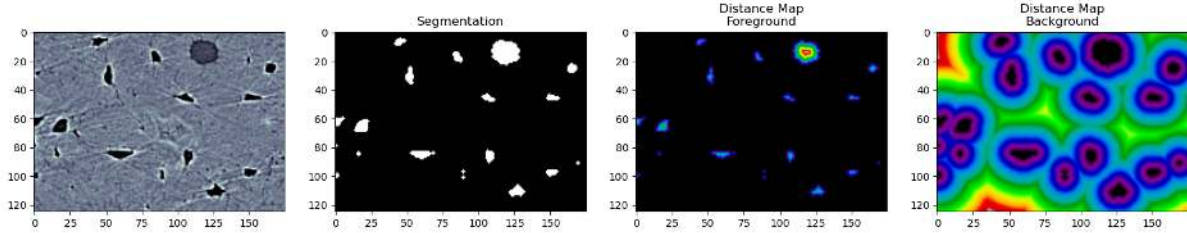
### 0.1.3 Previously on QBI ...

- Image Enhancement
- Understanding image histograms
- Automatic Methods
- Component Labeling
- Single Shape Analysis
- Complicated Shapes (Thickness Maps)

### 0.1.4 Learning Objectives

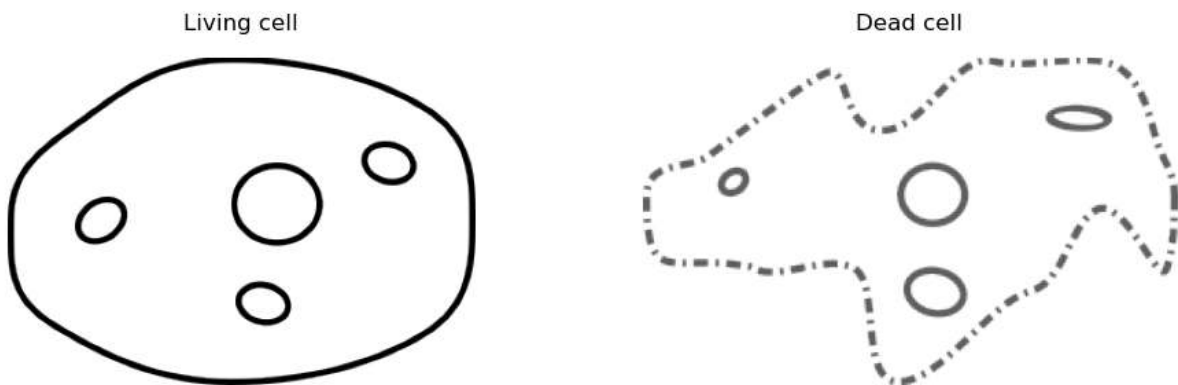
#### Motivation (Why and How?)

- How can we quantify the spatial distribution of particles?
- How can we extract topology of a structure?
- How do we identify separate objects when they are connected?
- How can we compare shape of complex objects when they grow?



## 0.1.5 Distribution Objectives - finding the right questions

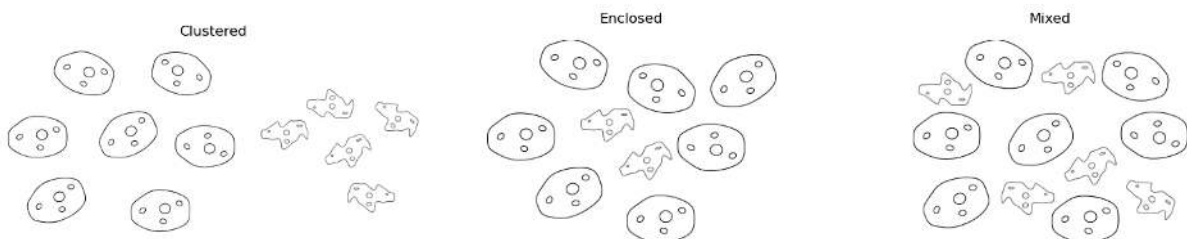
We want to know how many cells are alive



Maybe...

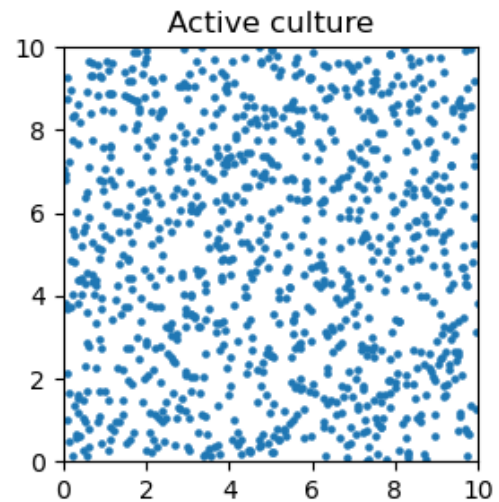
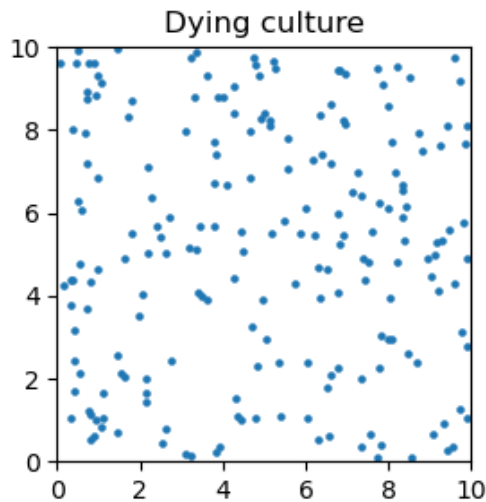
- small cells are dead and larger cells are alive → examine the volume distribution
- living cells are round and dead cells are really spiky and pointy → examine anisotropy

## Different distributions



### We want to know where the cells are alive or most densely packed

- We can visually inspect the sample (maybe even color by volume)
- We can examine the raw positions (x,y,z) but what does that really tell us?
- We can make boxes and count the cells inside each one
- How do we compare two regions in the same sample or even two samples?



### 0.1.6 So what do we still need

1. A way for counting cells in a region and estimating density without creating arbitrary boxes
2. A way for finding out how many cells are *near* a given cell, it's nearest neighbors
3. A way for quantifying how far apart cells are and then comparing different regions within a sample
4. A way for quantifying and comparing orientations

### What would be really great?

A tool which could be adapted to answering a large variety of problems

- multiple types of structures
- multiple phases

### 0.1.7 Metrics

We examined a number of different metrics in this lecture and additionally to classifying them as **Local** and **Global** we can define them as point and voxel-based operations.



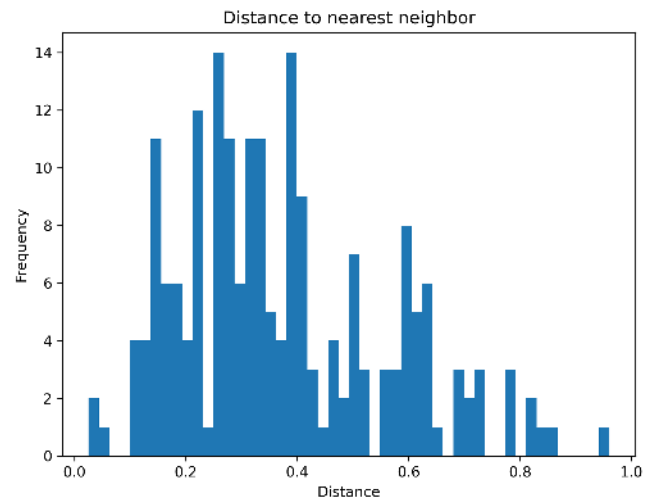
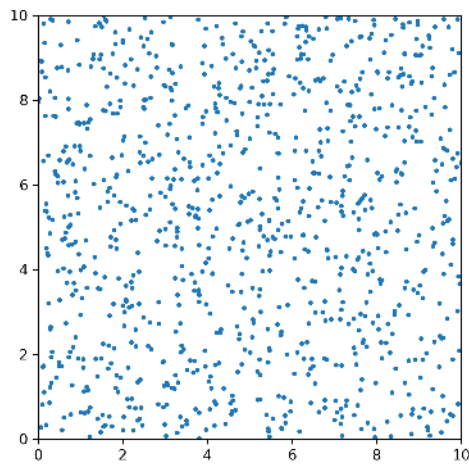
## Point Operations

- Nearest Neighbor
- Point (Center of Volume)-based Voronoi Tessellation
- Alignment

## Voxel Operation

- Voronoi Tessellation
- Neighbor Counting
- 2-point (N-point) correlation function

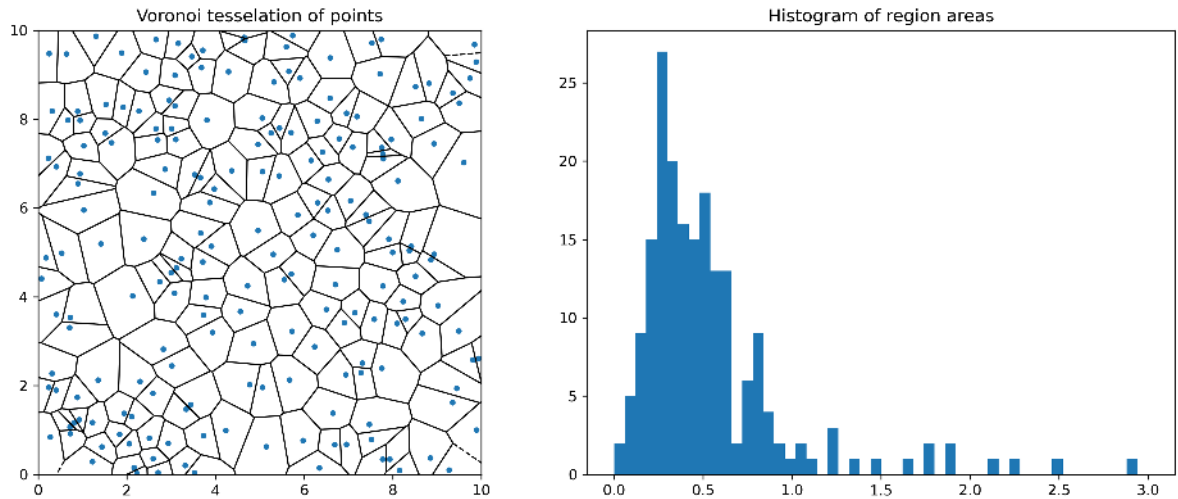
## Nearest neighbor distribution



## Voronoi tessellation

Voronoi tessellation is a method to compute the support area of regions in a point cloud. The support area is defined as the region where the distance to the dot is closer than any other dot in the cloud. This can be done either using the coordinated of the items or from images with dots. The distance map of the background can be used to compute the Voronoi tessellation.

The distribution of the region areas can be used to characterize the structure of the point cloud. Like in the histogram to the right.



### 0.1.8 Destructive Measurements

With most imaging techniques and sample types, the task of measurement itself impacts the sample.

- X-ray tomography which *claim* to be non-destructive still impart significant to lethal doses of X-ray radiation for high resolution imaging
- Electron microscopy, auto-tome-based methods, histology are all markedly more destructive and make longitudinal studies impossible

Even when such measurements are possible registration can be a difficult task and introduce artifacts

#### Why is this important?

- techniques which allow us to compare different samples of the same type.
- are sensitive to common transformations
- Sample B after the treatment looks like Sample A stretched to be 2x larger
- The volume fraction at the center is higher than the edges but organization remains the same

## 0.2 Structure analysis

One main objective in scientific imaging is to describe and quantify shapes.

We have already looked into several metrics to describe items in the image:

- Area
- Perimeter
- Orientation
- Position
- etc.

Today, we will look into two further techniques used for structure analysis:

- Skeletons
- Advanced object labeling

## 0.3 Skeletonization / Networks

Thin structures and networks often appears in images.

### 0.3.1 Thin structures to analyze

The world is full of thin structures we want to analyze. Below you see some examples.



### 0.3.2 What we want to know about networks

In many cases we want to describe the topology of the network

- which structures are connected
- how they are connected
  - Are there loops
- express the network in a simple manner
- quantify *tortuosity*
- branching

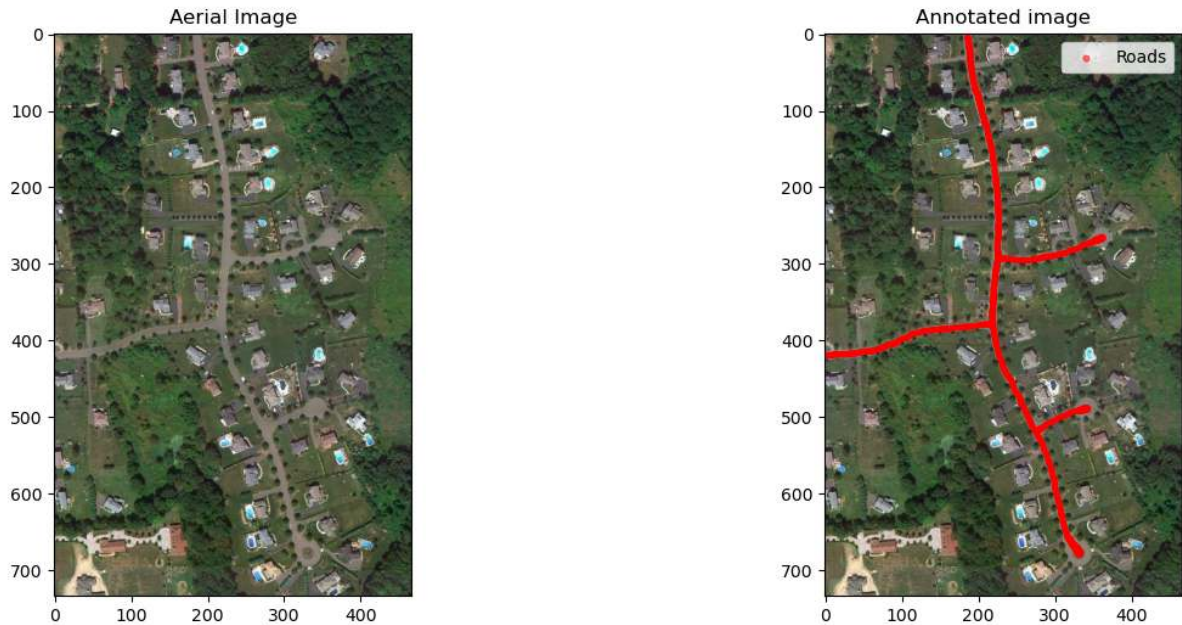
We start with a simpler example from the EPFL Dataset: EPFL CVLab's Library of Tree-Reconstruction Examples (<http://cvlab.epfl.ch/data/delin>)

### 0.3.3 To describe network topology we need

The minimal structure that spans the structure topology - i.e. a skeleton

## 0.4 Network analysis explained in real images

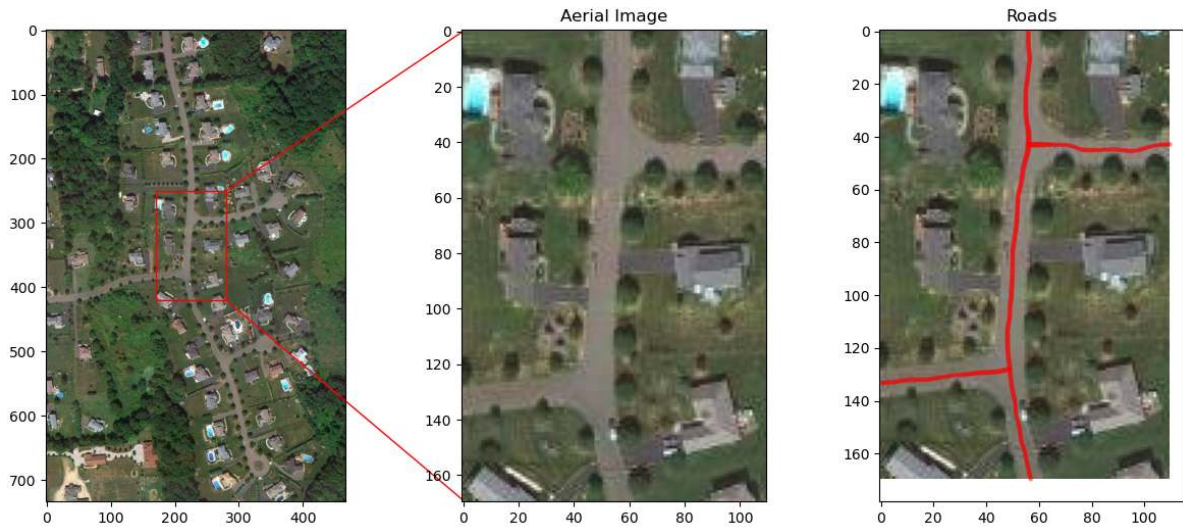
We will use an aerial picture of a street as test image for exploring how to build skeleton and to improve the performance of the skeletonization. We also have markup data as ground truth for the position of the street.



### 0.4.1 A close-up of the street

The full street image is too large to see all details. Therefore, we crop a piece in middle of the picture.

```
# Crop image
im_crop = im_data[250:420:1, 170:280:1]
# Select street point in cropped region
mk_crop = mk_data.query('y>=250').query('y<420').query('x>170').query('x<=280').copy()
# Adjust position. bias
mk_crop.x = (mk_crop.x-170)
mk_crop.y = (mk_crop.y-250)
```



## 0.4.2 Let's try finding the roads

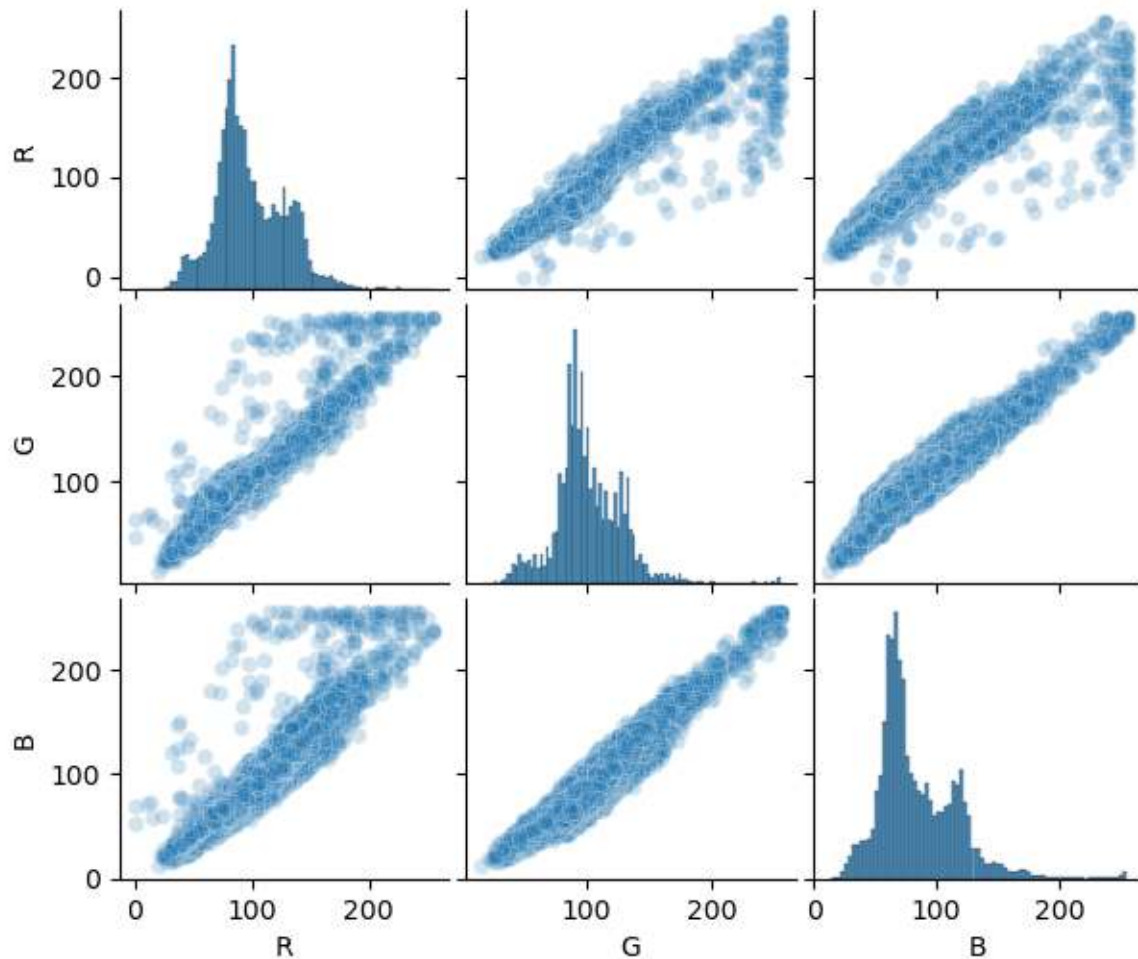
The first thing we have to do for our street analysis example is to identify the street among all other features in the picture.

### Step 1: Inspecting the data

Let's first get acquainted with the data using a pair plot for the color channels RGB in the image

```
from sklearn.cluster import KMeans
from seaborn import pairplot
import pandas as pd
img = im_crop
lst = {'R': img[:, :, 0].ravel(), 'G': img[:, :, 1].ravel(), 'B': img[:, :, 2].ravel()}
dfRGB = pd.DataFrame(lst)

pp=pairplot(dfRGB, plot_kws={'alpha': 0.2}); pp.fig.set_size_inches(6, 5)
```



### Trying a different color space - HSV

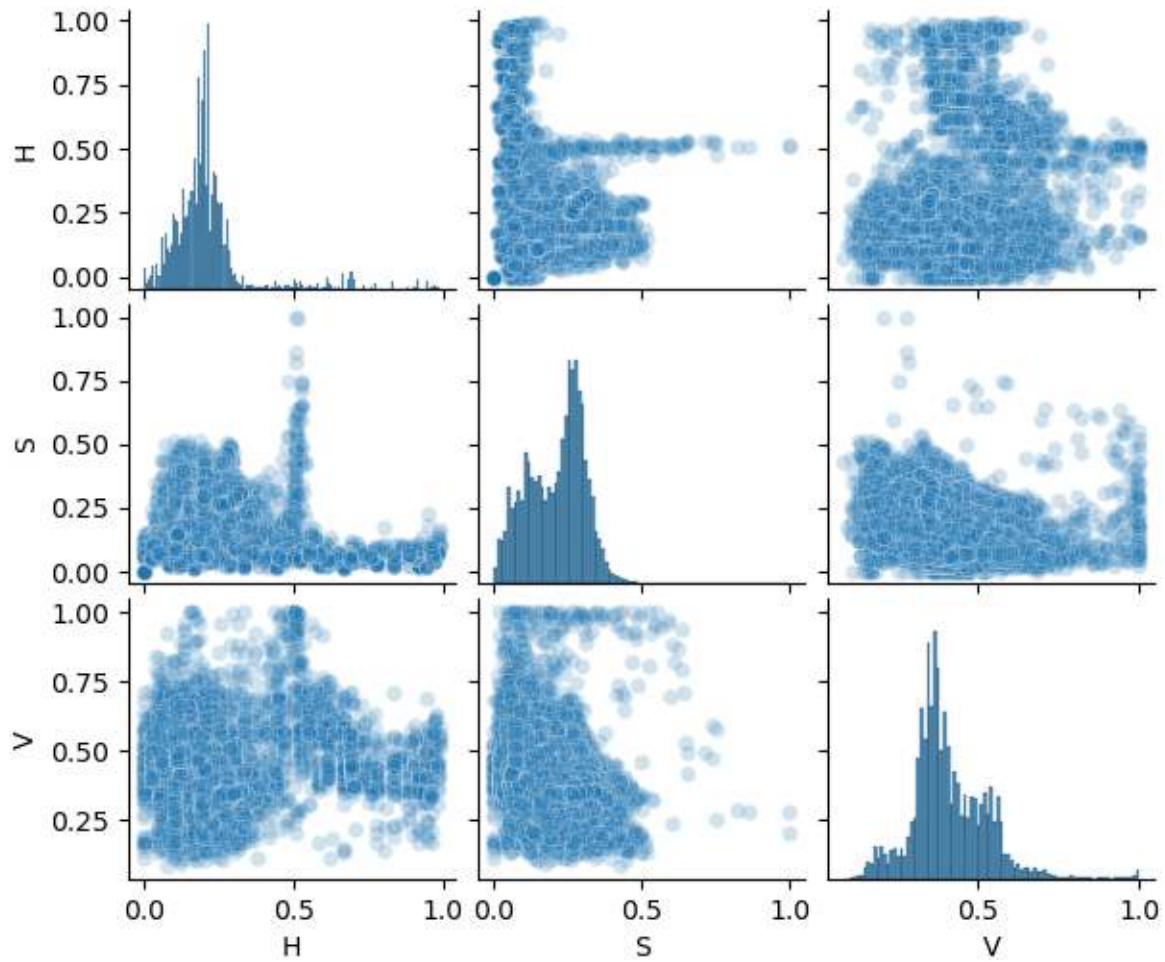
Color images can be represented by several color space models. Most image files use the RGB space that separates the colors into three color channels red, green, and blue. An alternative color space is described by an angle that scans color spectrum (hue), how intense the color is (saturation), and the intensity (value). More about this on [Wikipedia](https://en.wikipedia.org/wiki/HSL_and_HSV).

The intensity is essentially the average values of the RGB color channels.

```
hsv_img = rgb2hsv(img)
lst = {'H': hsv_img[:, :, 0].ravel(), 'S': hsv_img[:, :, 1].ravel(), 'V': hsv_img[:, :, 2].
      .ravel()}
dfHSV = pd.DataFrame(lst)

pp=pairplot(dfHSV, plot_kws={'alpha': 0.2}); pp.fig.set_size_inches(6, 5)
```





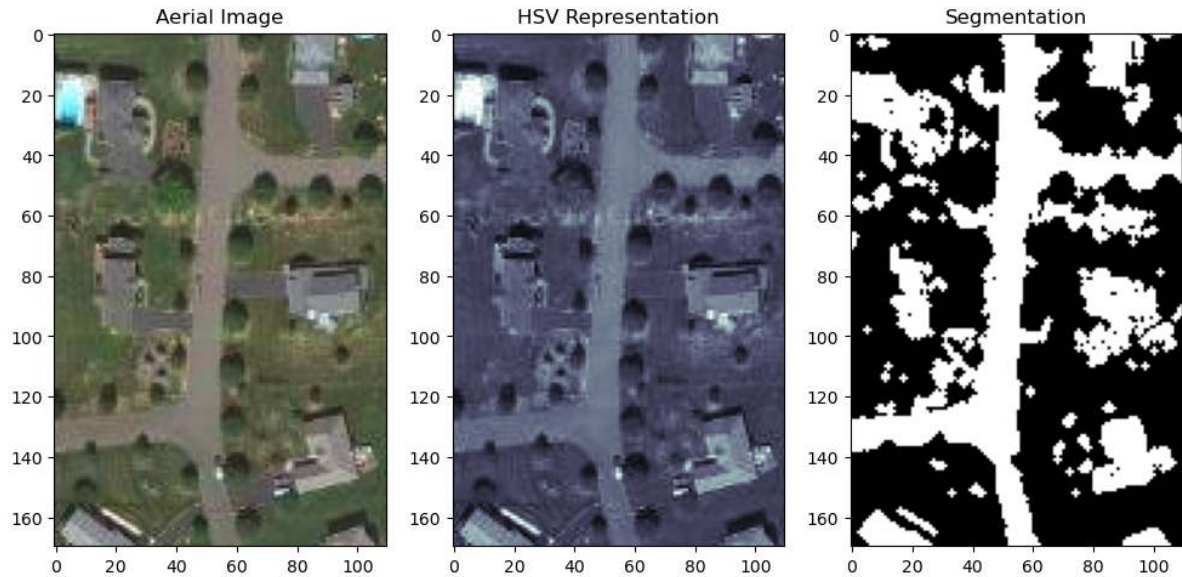
## Step 2: Segmentation

### Thresholding

The picture is a color image that uses the RGB color model. In this case we will convert the color model to HSV and use the V parameter for the thresholding. An empirical value for the threshold is  $v > 0.4$ .

```
def thresh_image(in_img):
    v_img = rgb2hsv(in_img)[: , : , 2]
    th_img = v_img > 0.4
    op_img = opening(th_img, disk(1))
    return op_img
```

```
seg_img = thresh_image(im_crop)
```

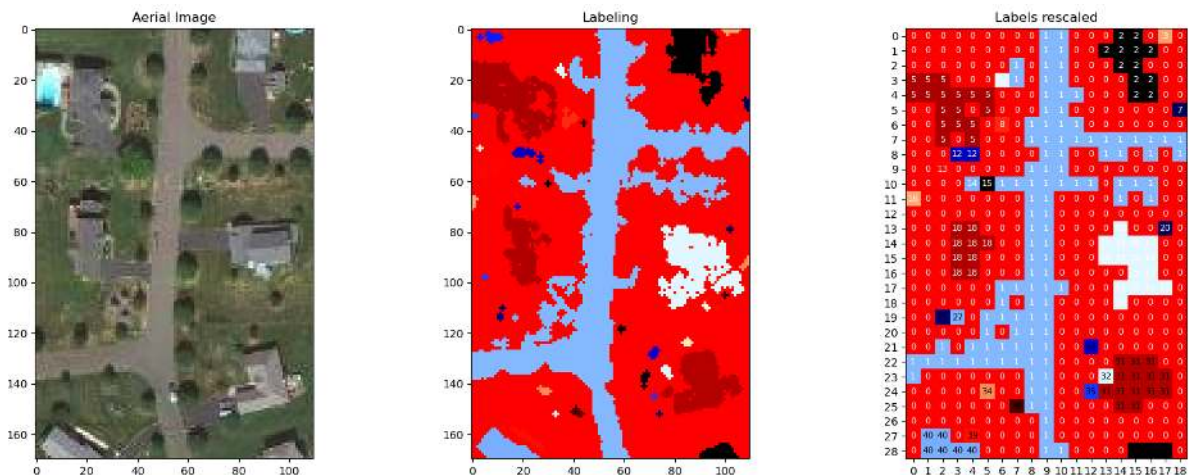


The chosen threshold is able to identify the street but there are also many structures on the sides. This is expected from a single threshold. The roofs of the houses have about the same gray level as the street.

### Step 3: Identify structures

Using connected component labeling (lecture 6)

```
lab_img0 = label(seg_img)
```



The labeling identifies the disconnected structures in the image which we show with different colors in the labeled image. The panel to the right was downsamples to allow us to show the labels on top of the labeled regions. The street appears to be assigned label '1'.

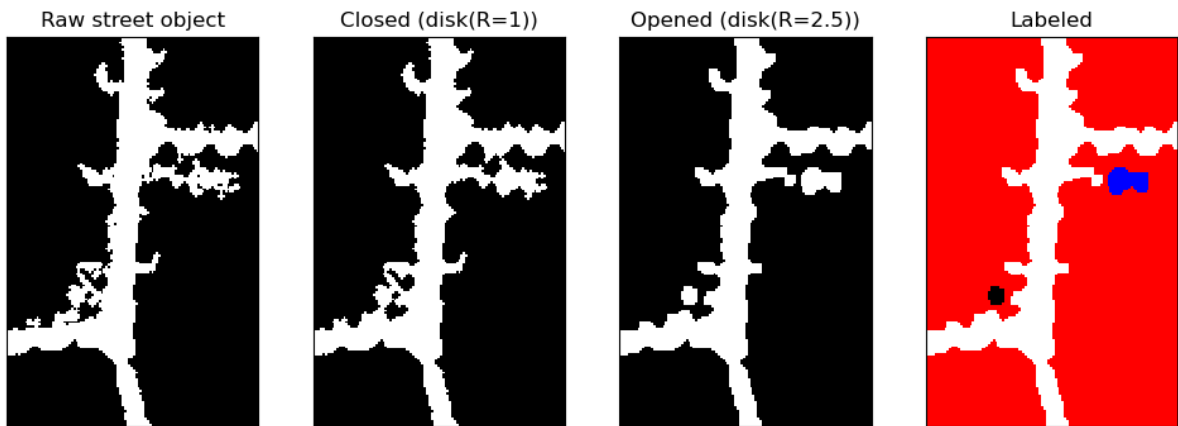


## Step 4: Cleaning up

In the next step we extract the street pixels and apply some morphological cleaning operations (closing and opening). The opening operation disconnected some structures from the street. Therefore, we use labeling a second time to remove all structure but the largest one, the street object.

```
from skimage.morphology import opening, closing, disk

street = (lab_img0==1)
clean1 = closing(street, footprint=disk(1))
clean2 = opening(clean1, footprint=disk(2.5))
# clean1 = opening(street, footprint=disk(3))
# clean2 = closing(clean1, footprint=disk(1))
lab_img = label(clean2)
```



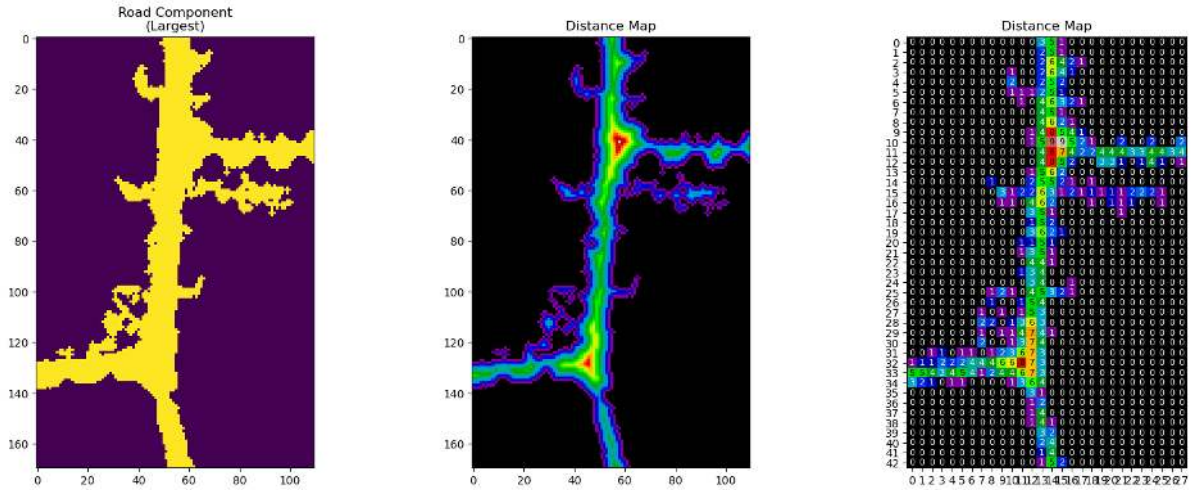
## 0.5 Skeletonization - take one

The previous steps prepared the structure we want to analyze. Now, we will explore some methods to extract the skeleton. This is essentially the structure drawn by the largest distance to the edge. Therefore,

The first step is to take the distance transform the structure

$$I_d(x, y) = \text{dist}(I(x, y))$$

```
from scipy import ndimage
keep_lab_img = (lab_img0 == 1) # Create an image with pixels belonging to sement i
dist_map = ndimage.distance_transform_edt(keep_lab_img.astype(float))
```



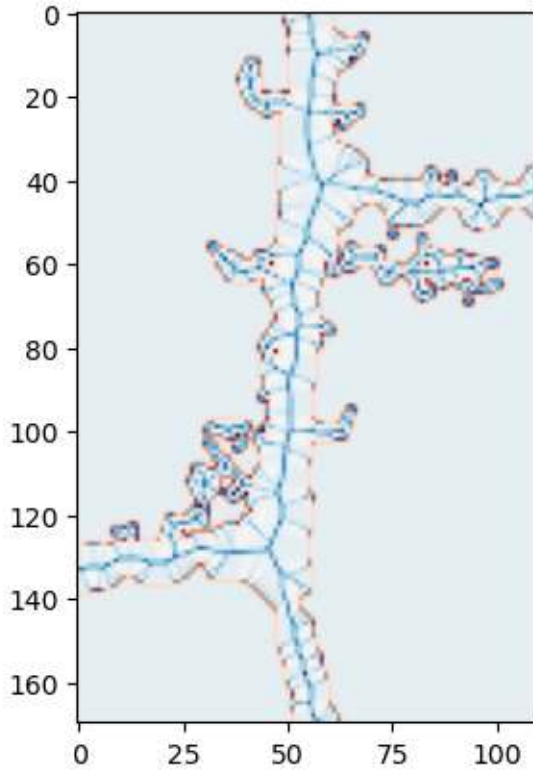
We can see in this image there are already local maxima that form a sort of backbone which closely maps to what we are interested in.

## 0.5.1 Skeletonization: Ridges

By using the Laplacian filter as an approximate for the derivative operator which finds the values which high local gradients.

$$\nabla I_d(x, y) = \left( \frac{\delta^2}{\delta x^2} + \frac{\delta^2}{\delta y^2} \right) I_d \approx \underbrace{\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}}_{\text{Laplacian Kernel}} * I_d(x, y)$$

```
lapskel = laplace(dist_map)
```



The Laplacian provides strong signals when there is an abrupt change in gradient direction. This happens in two regions of the distance map

- The boundary to the background which is irrelevant in this case, and
- At the medial axis of the object where there is a local maximum ridge.

## 0.5.2 Creating the skeleton

We can locate the local maxima of the structure by setting a minimum surface distance

$$I_d(x, y) > MIN_{DIST}$$

and combining it with a minimum slope value

$$\nabla I_d(x, y) > MIN_{SLOPE}$$

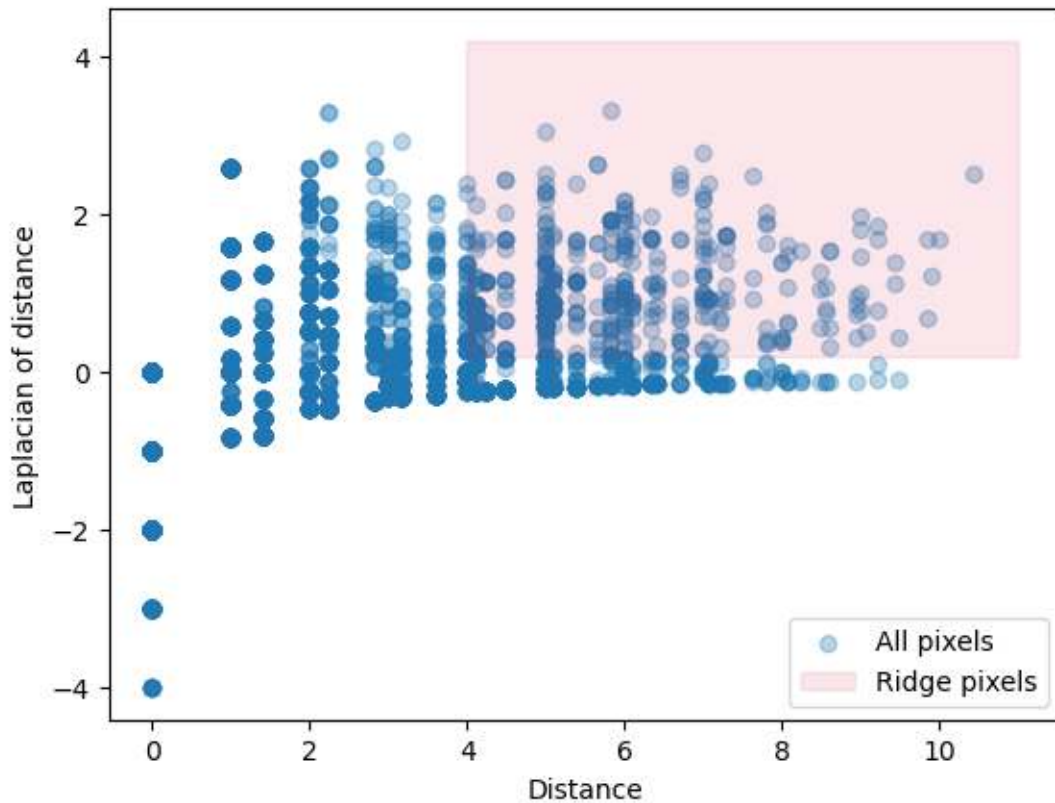
### Thresholds on the distance map

Harking back to our earlier lectures, this can be seen as a threshold on a feature vector representation of the entire dataset.

- We first make the dataset into a tuple

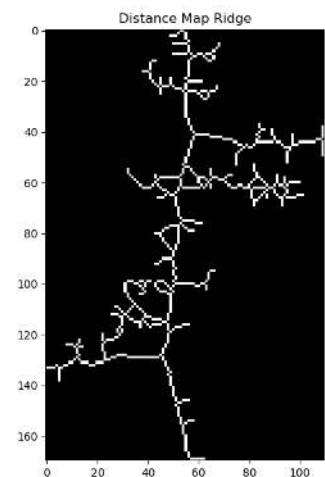
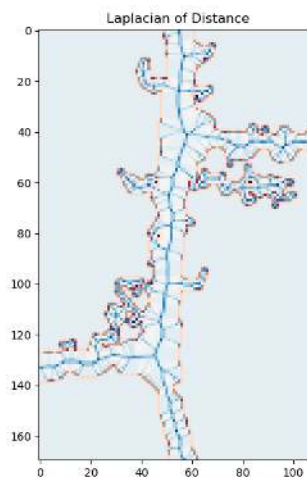
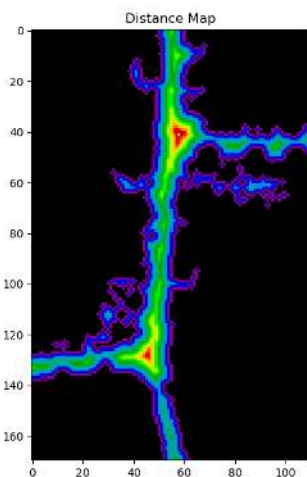
$$cImg(x, y) = \langle \underbrace{I_d(x, y)}_1, \underbrace{\nabla I_d(x, y)}_2 \rangle$$

$$skelImage(x, y) = \begin{cases} 1, & cImg_1(x, y) \geq MIN_{DIST} \text{ and } cImg_2(x, y) \geq MIN_{SLOPE} \\ 0, & \text{otherwise} \end{cases}$$



### Resulting ridge skeleton

```
lapskel = laplace(dist_map)
skel    = medial_axis(keep_lab_img, return_distance=False) ; # we use medial axis
↳ since it is cleaner
```



This already looks like a convincing skeleton of the street object. It may be a bit cluttered with unmotivated branches, but it could be a good starting point.

## 0.6 Skeletonization with Morphological thinning

From scikit-image documentation ([http://scikit-image.org/docs/dev/auto\\_examples/edges/plot\\_skeleton.html](http://scikit-image.org/docs/dev/auto_examples/edges/plot_skeleton.html))

Morphological thinning, implemented in the `thin` function, works on the same principle as `skeletonize`:

- remove pixels from the borders at each iteration until none can be removed without altering the connectivity.
- The different rules of removal can speed up skeletonization and result in different final skeletons.

The `thin` function also takes an optional `max_iter` keyword argument to limit the number of thinning iterations, and thus produce a relatively thicker skeleton.

We can use this to thin the tiny junk elements first then erode, then perform the full skeletonization

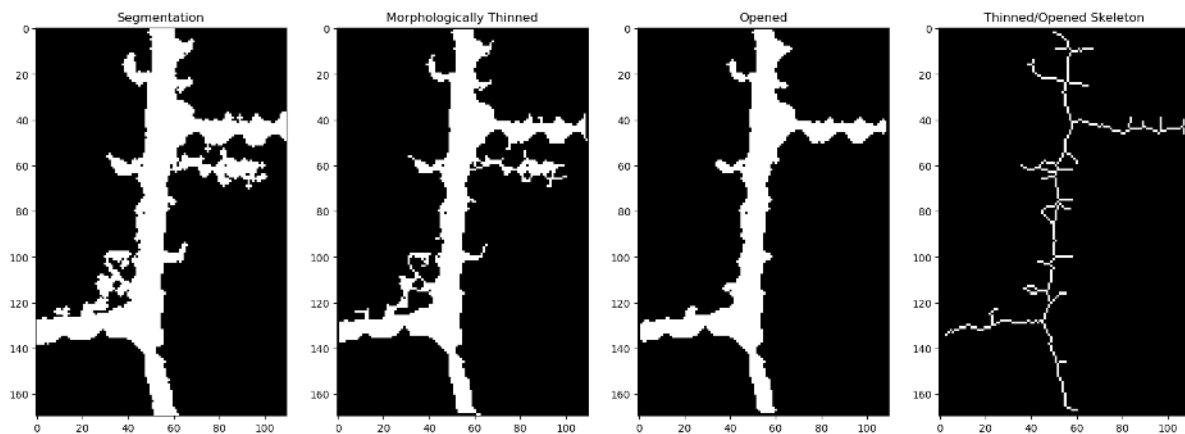
### 0.6.1 Try morphological thinning

We saw in the previous example based on the Laplacian of the distance map that there were too many branches. Let's try to reduce them by applying some more cleaning.

In the following example we apply the sequence of

1. An erosion, here with one iteration of the thinning operation
2. A morphological opening, which removes the connection of some of the blobs.
3. And once more filtering object to keep the street only.
4. Use the medial axis skeletonization algorithm.

```
from skimage.morphology import thin, erosion
thin_image = thin(keep_lab_img, max_num_iter=1)
er_thin_image = opening(thin_image, disk(1))
er_thin_image = label(er_thin_image) == 1
opened_skeleton = medial_axis(er_thin_image, return_distance=False)
```



Now we got one step closer to a clean skeleton, but it can be even further improved.

### Still overgrown

The skeleton is still problematic for us and so we require some additional improvements to get a perfect skeleton.

There are a lot spurious branches and even loops on the thinned skeleton. These need to be pruned to get a skeleton that correspond to our expectations.

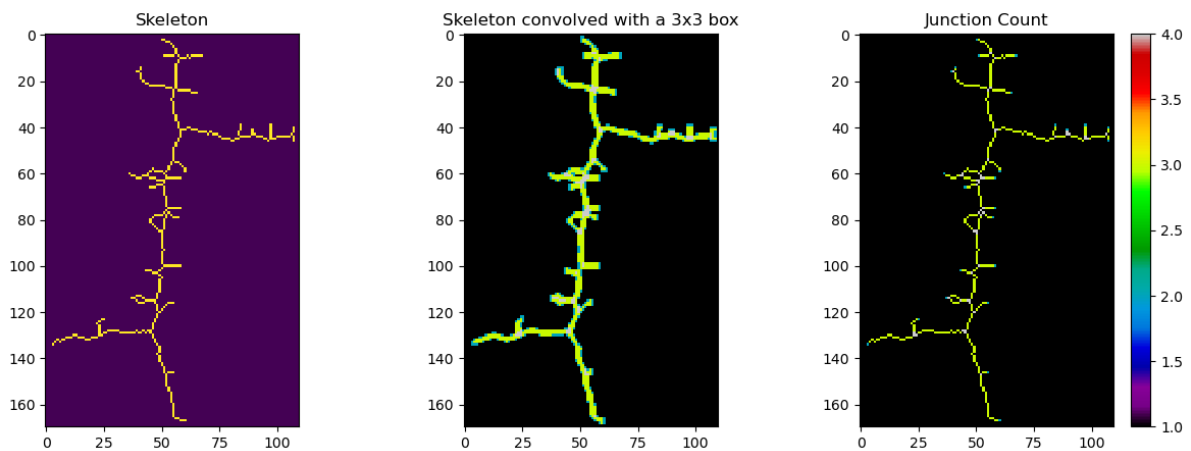
## 0.7 Skeleton: Junction Overview

With the skeleton which is ideally one voxel thick, we can characterize the junctions in the system by looking at the neighborhood of each voxel.

Junctions are the pixels where more than two branches intersect. In 2D, there can be at most four branches meeting up at a single pixel. There can however be clusters of junctions that correspond to more complex junction topology.

Here, we will use the convolution with a 3x3 box kernel to identify the junctions. It essentially sums all pixels in the neighborhood. The convolution widens the skeleton by one pixel in all directions. This wider skeleton can be masked with the original skeleton.

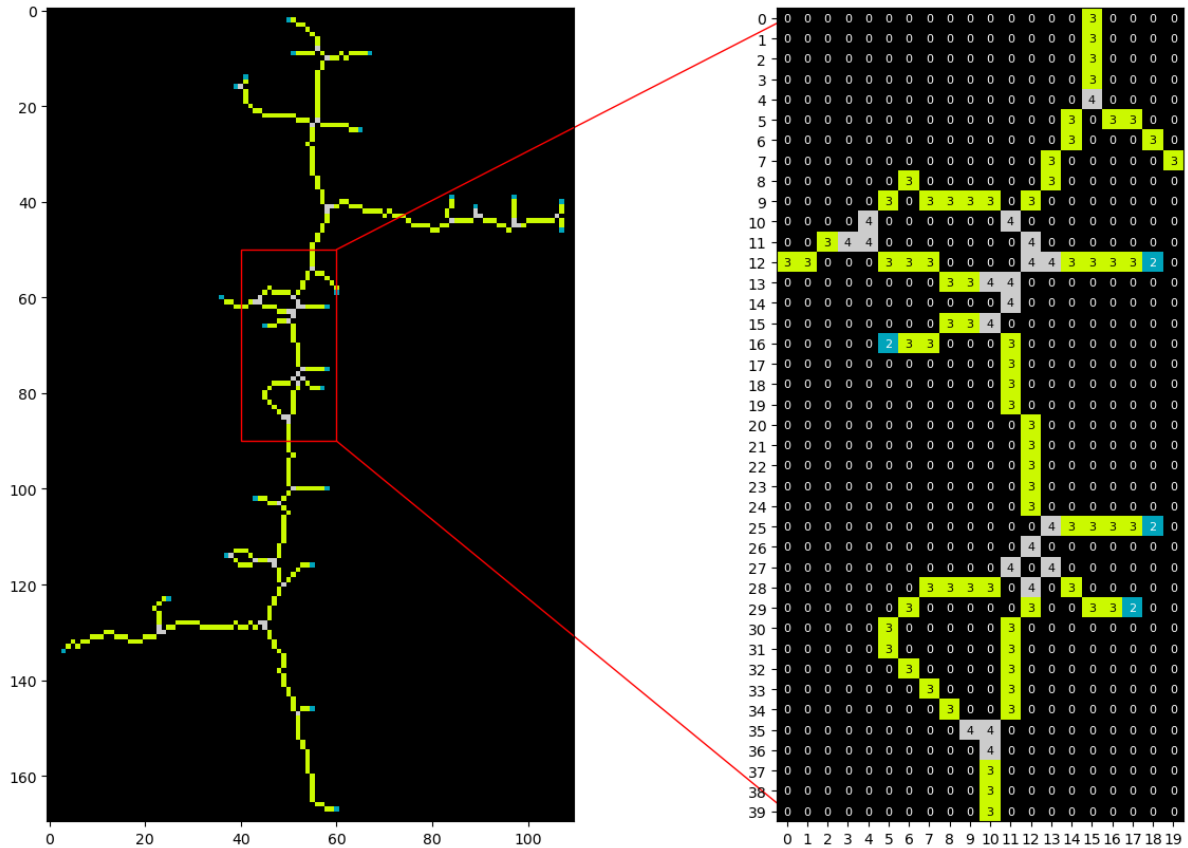
```
k=np.ones((3,3))
neighbor_conv = convolve(opened_skeleton.astype(int), k)
# Masking skeleton
neighbor_conv_masked = neighbor_conv.copy()
neighbor_conv_masked[~opened_skeleton] = 0
```



The box kernel counts all pixels in the neighborhood including the central pixel. Thus, the junction count will be number of neighbors + 1. The kernel could be modified by replacing the central element by '0'. This would have the effect that the central pixel is not included in the sum and the value would be the number exact number of neighbors. A caveat of this kernel is that then it would not be possible to identify singular pixels.

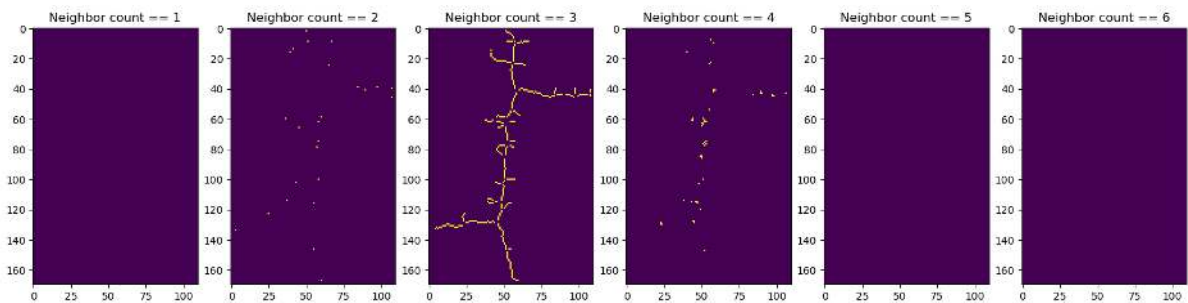
### 0.7.1 Close-up on the skeleton

The skeleton pixels of the filteres and masked skeleton has different values. These values tell how many neighbor pixels each pixel has.



### 0.7.2 Skeleton pixel neighborhood classes

Now that we have seen that the convolution gives information about the neighborhood constellation, we can start to look for different characteristic combinations.



In the table below we make a rough categorization of the neighbor counts.

| Pixel counts | Interpretation     |
|--------------|--------------------|
| 0            | Background         |
| 1            | Isolated point     |
| 2            | End point          |
| 3            | Line segment       |
| 4            | Three-way Junction |
| 5            | Four-way junction  |

### 0.7.3 Smarter kernel coding

The uniform kernel was able to guide us to right positions on the skeleton. There are however ambiguous cases that will be interpreted in the wrong way. This can be handled by using a kernel that keeps track of the exact configuration of the pixel. The idea is to use neighbor weight from the positions in a binary number i.e.  $2^N$  with  $N \in [0 \dots 9]$

Using a kernel like this:  $j_4 =$ 

|   |     |   |
|---|-----|---|
|   | 1   |   |
| 8 | 256 | 2 |
|   | 4   |   |

 $j_8 =$ 

|     |     |    |
|-----|-----|----|
| 1   | 2   | 4  |
| 128 | 256 | 8  |
| 64  | 32  | 16 |

 $\$$

The neighborhood is uniquely coded, less ambiguous than summing pixels.

- Coding pixels with values  $x$ :
  - $x = 0$  are background
  - $x < 256$  touch the skeleton
  - $x > 256$  are on the skeleton
- The number of branches and their orientation can be encoded by counting bit flips or using a LUT.

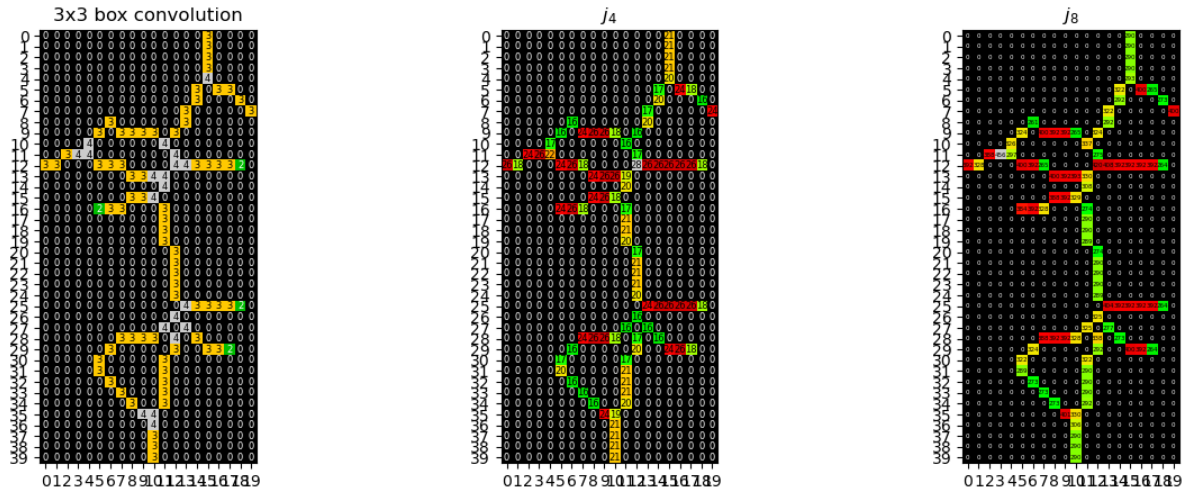
This coding is orientation sensitive, which may be too detailed for some applications.

### Compare different skeleton analysis kernels

```
n_crop = neighbor_conv_masked[50:90, 40:60]

neighbor_j4 = convolve(opened_skeleton[50:90, 40:60].astype(int), np.array([[0,1,0],
↪ [8,16,2], [0,4,0]]))
neighbor_j4[opened_skeleton[50:90, 40:60]==0]=0
neighbor_j8 = convolve(opened_skeleton[50:90, 40:60].astype(int), np.array([[1,2,4],
↪ [128,256,8], [64,32,16]]))
neighbor_j8[opened_skeleton[50:90, 40:60]==0]=0
```





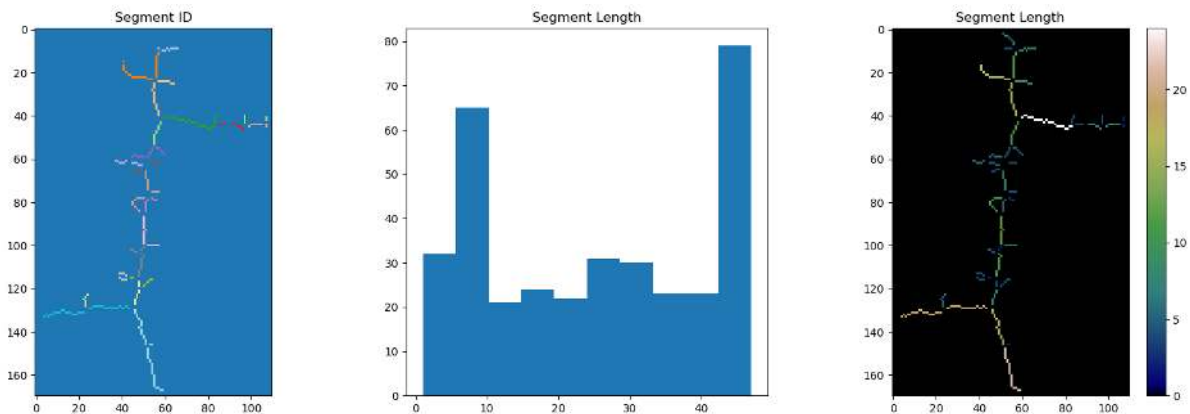
## 0.8 Dedicated pruning algorithms

- Ideally model-based
- Minimum branch length (using component labeling on the Count==3)
- Minimum branch width (using the distance map values)

### 0.8.1 Analyzing segment length

```
lab_seg = label(neighbor_conv_masked == 3)

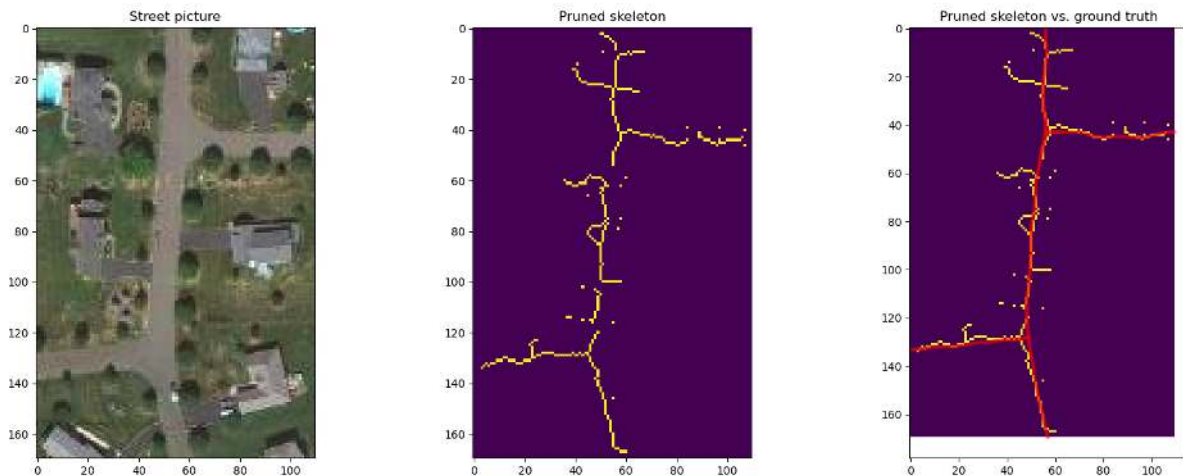
label_length_img = np.zeros_like(lab_seg)
for i in np.unique(lab_seg[lab_seg > 0]):
    label_length_img[lab_seg == i] = np.sum(lab_seg == i)
```



### Looking at the pruned skeleton

The skeleton should be pruned to only contain line segments with more than five pixels. This is done by thresholding the `length_skeleton` variable. The end-points and junctions are added to get a complete skeleton again. These pixels are picked from the convolved image, value '2' for end-points and any value >3 is for the endpoints.

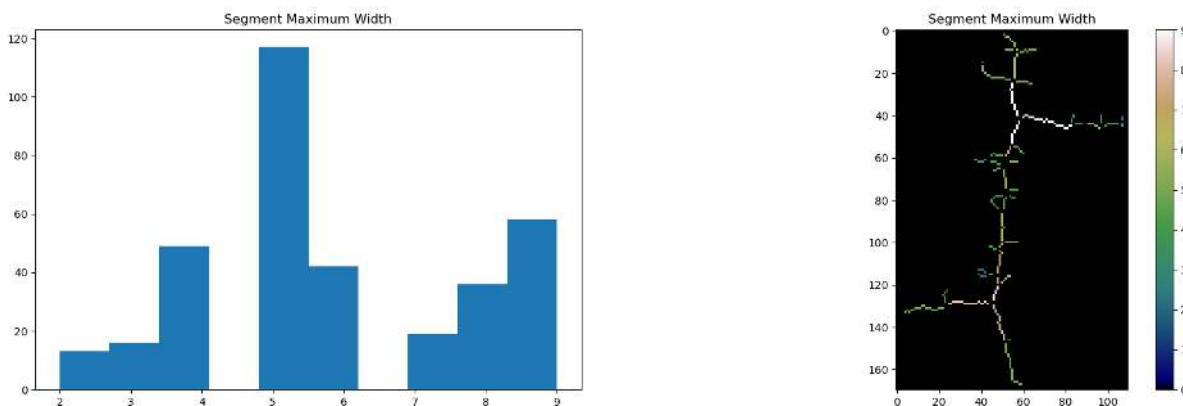
```
length_skeleton = (label_length_img > 5) + \
    (neighbor_conv_masked == 2) + \
    (neighbor_conv_masked > 3)
```



### 0.8.2 Analyzing maximum segment width

The segment width is a different metric for the skeleton pruning. The width is provided by computing the distance map of the original structure. Each segment can now be assigned the greatest distance found at the pixels belonging to that segment.

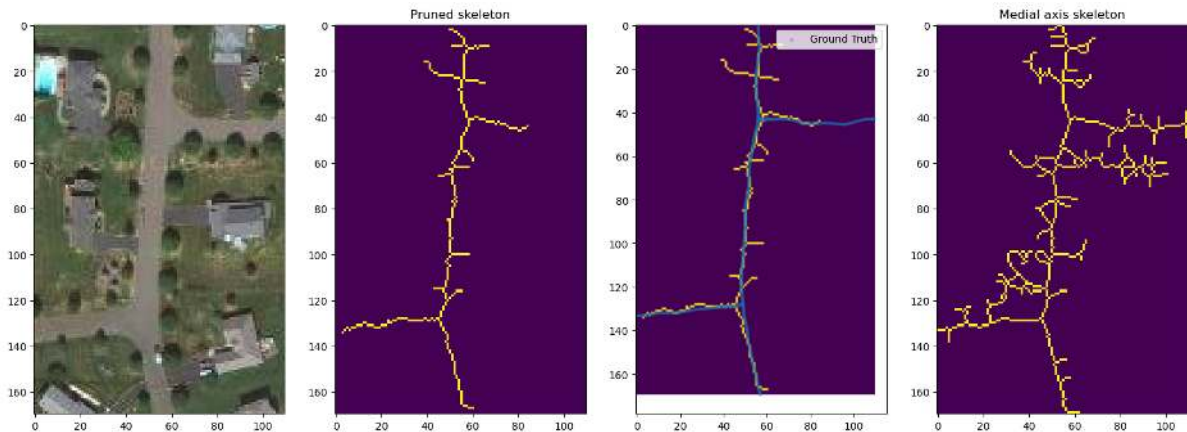
```
label_width_img = np.zeros_like(lab_seg)
for i in np.unique(lab_seg[lab_seg > 0]):
    label_width_img[lab_seg == i] = np.max(dist_map[lab_seg == i])
```



## Pruning using structure width

Pruning for the structure width, we use a threshold on the segment maximum width skeleton. Like for the length pruning, we again add the junctions and the end points.

```
width_skeleton = (label_width_img > 4.5) \
    + (neighbor_conv_masked == 2) \
    + (neighbor_conv_masked > 3)
width_skeleton = label(width_skeleton) == 1
```



### 0.8.3 An alternative

We know clean data is easier to work with...

Let's try to improve the input before creating the skeleton:

- Change segmentation method
- Use morphological filters

### Other ways to do the segmentation...

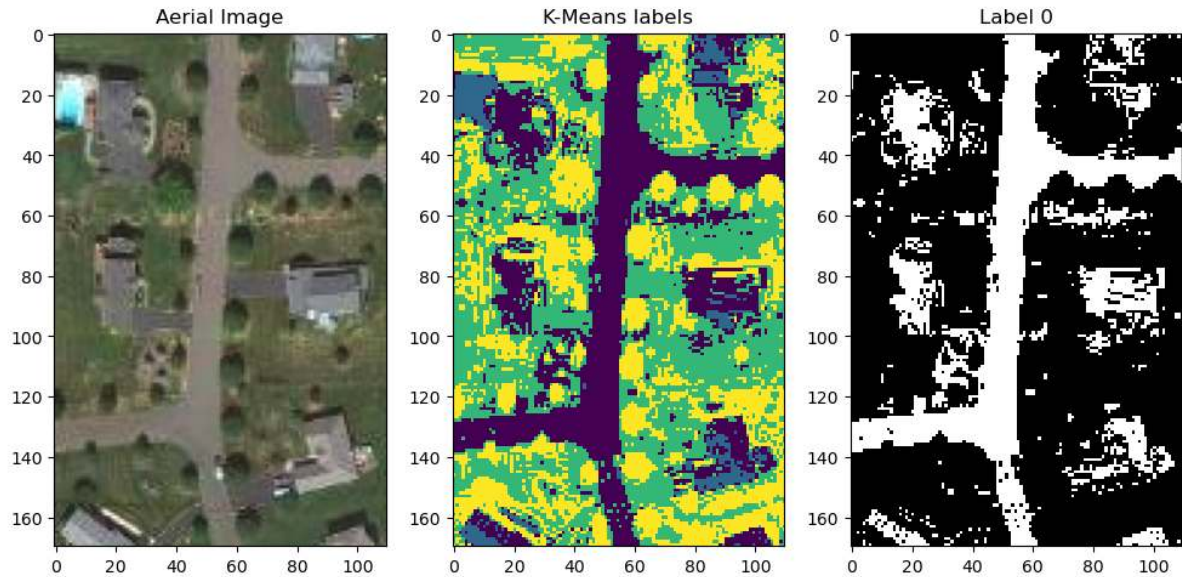
We used a color space transformation

We could also use:

- Unsupervised segmentation like k-means
- Supervised segmentation like k-Nearest Neighbors (requires training)

Let's try k-Means clustering on the RGB channels

```
km=KMeans(n_clusters=4, random_state=42)
km.fit(dfRGB[['R', 'B']])
km_img = (km.labels_.reshape(im_crop.shape[:2]) == 0)
```



### Cleaning up

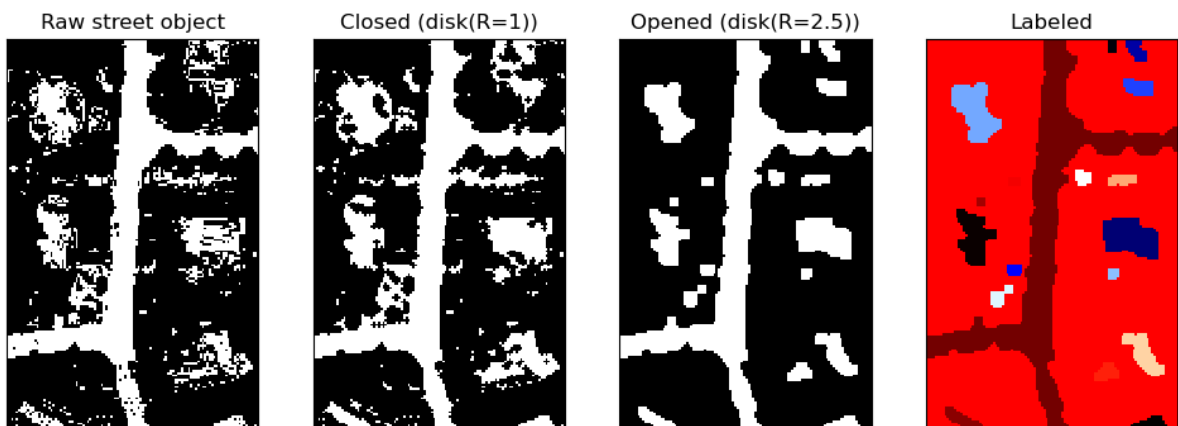
We know from lecture 4 that morphological filtering can clean up messy segmentations with many misclassified pixels. In this case, we will use a sequence of closing and opening to improve the image before labelling to select the street object.

Note that the structure elements are of different sizes here. This is because the size of the false negatives is different from the false positives. Also the order of the opening and closing operations matter. In this particular case it made sense to apply closing before opening. Which depends if you create or destroy structures or connections and must be tested.

```
from skimage.morphology import opening, closing, disk

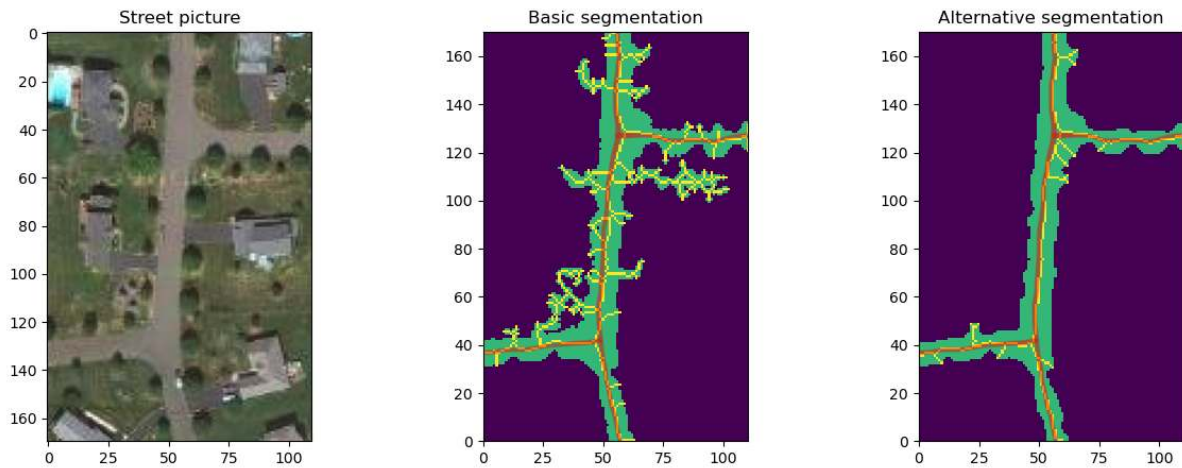
street = (km_img==1)
clean1 = closing(street, footprint=disk(1))
clean2 = opening(clean1, footprint=disk(2.5))

lab_img = label(clean2)
```



## Make skeleton

```
new_skeleton = medial_axis(lab_img==1, return_distance=False)
```



You can clearly see that the alternative approach creates a less ambiguous structure and the following skeleton is much leaner. The leaner skeleton requires less post processing. Please note that the basic segmentation would also lead to a much leaner skeleton. This was not done in the lecture because we wanted to demonstrate the skeleton analysis which many loops and spurious branches.

## 0.9 Establish Topology

From the cleaned, pruned skeleton we can start to establish topology.

Using the same criteria as before we can break down the image into

- segments,
- junctions,
- and end-points



### 0.9.1 Topology of the street network



### 0.9.2 Getting Topology in Image Space

We want to determine which nodes are directly connected in this image so we can extract a graph. If we take a simple case of two nodes connected by one edge and the bottom node connected to another edge going nowhere.

We have an image with edges and nodes:  $\$ \begin{bmatrix} n & 0 & 0 & 0 \\ 0 & e & 0 & 0 \\ 0 & 0 & n & e \end{bmatrix} \$$

We can use component labeling to identify each node and each edge uniquely

## Node Labels

$$N_{lab} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 \end{bmatrix}$$

## Edge Labels

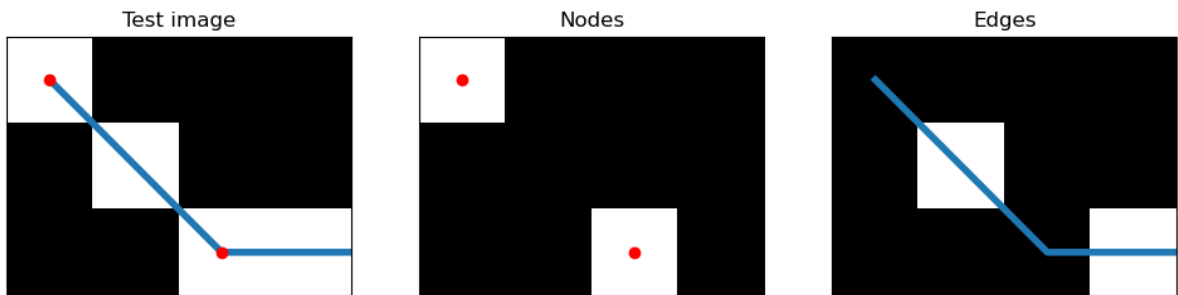
$$E_{lab} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 2 \end{bmatrix}$$

We can then use a dilation operation on the nodes and the edges to see which overlap

## Small topology analysis example

In this example we create images with nodes and edges respectively. The items in the images are labelled to tell which analyze. The next step is to dilate each node in the image to tell which edges it is connected to. An edge belongs to a node if there is an overlap between the dilated node (its neighborhood) and any pixel of the edge. The edges in this example are very short, only one pixel. This is normally not the case.

```
from skimage.morphology import dilation
n_img = np.zeros((3, 4))
e_img = np.zeros_like(n_img)
n_img[0, 0] = 1
e_img[1, 1] = 1
n_img[2, 2] = 1
e_img[2, 3] = 1
```

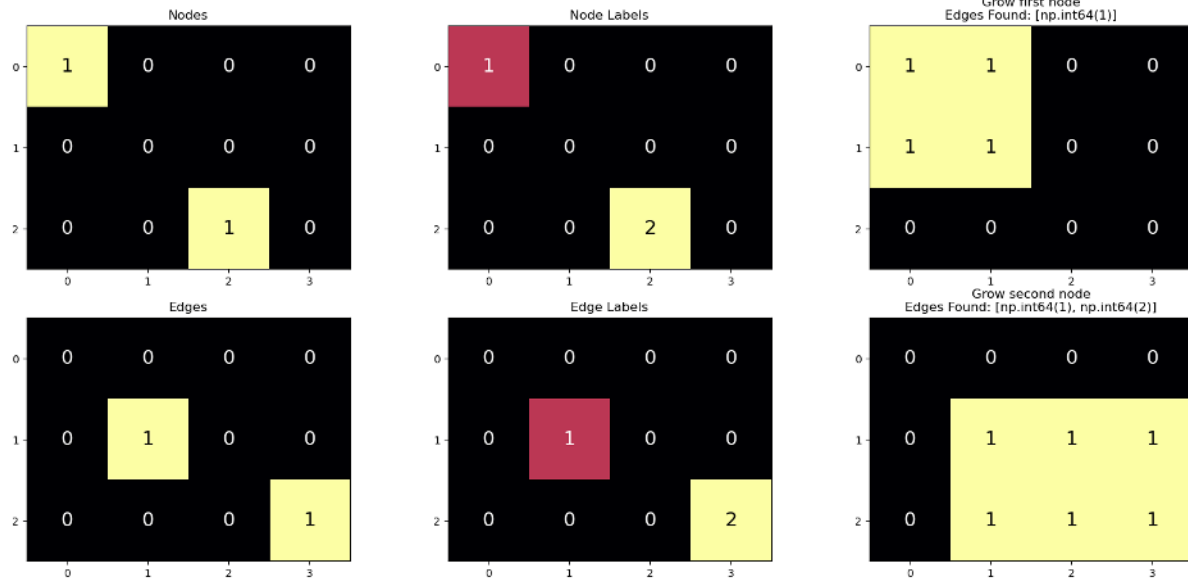


## The resulting analysis

Here, you can see the result of the edge-node connection code.

```
n_labs = label(n_img)
e_labs = label(e_img)

n_grow_1 = dilation(n_labs == 1, np.ones((3, 3)))
n_grow_2 = dilation(n_labs == 2, np.ones((3, 3)))
```



Here, you see that node 1 only overlaps edge 1 while node 2 overlaps both edges 1 and 2 as it is between them.

### Analysing the topology of the street network

Using skeleton pixel neighborhood analysis with a 3x3 kernel as before we have

| Pixel counts | Interpretation     | Category              |
|--------------|--------------------|-----------------------|
| 0            | Background         | Background/Don't care |
| 1            | Isolated point     | Don't care            |
| 2            | End point          | Node                  |
| 3            | Line segment       | Edge                  |
| 4            | Three-way Junction | Node                  |
| 5            | Four-way junction  | Node                  |

### Identify edges and nodes

The first step of the analysis is to label nodes and edges. This is done by using the connected components labeling of the neighborhood filtered image.

1. The nodes are identified by neighborhood counts greater than 3.
2. The edges are identified as neighborhood counts equal to 3.

These two images will be labelled to give the segments id numbers.

The next step is to create a dictionary of all nodes containing the CoG and the average width at the node.

```
node_id_image = label((ws_neighbors > 3) | (ws_neighbors == 2))
edge_id_image = label(ws_neighbors == 3)

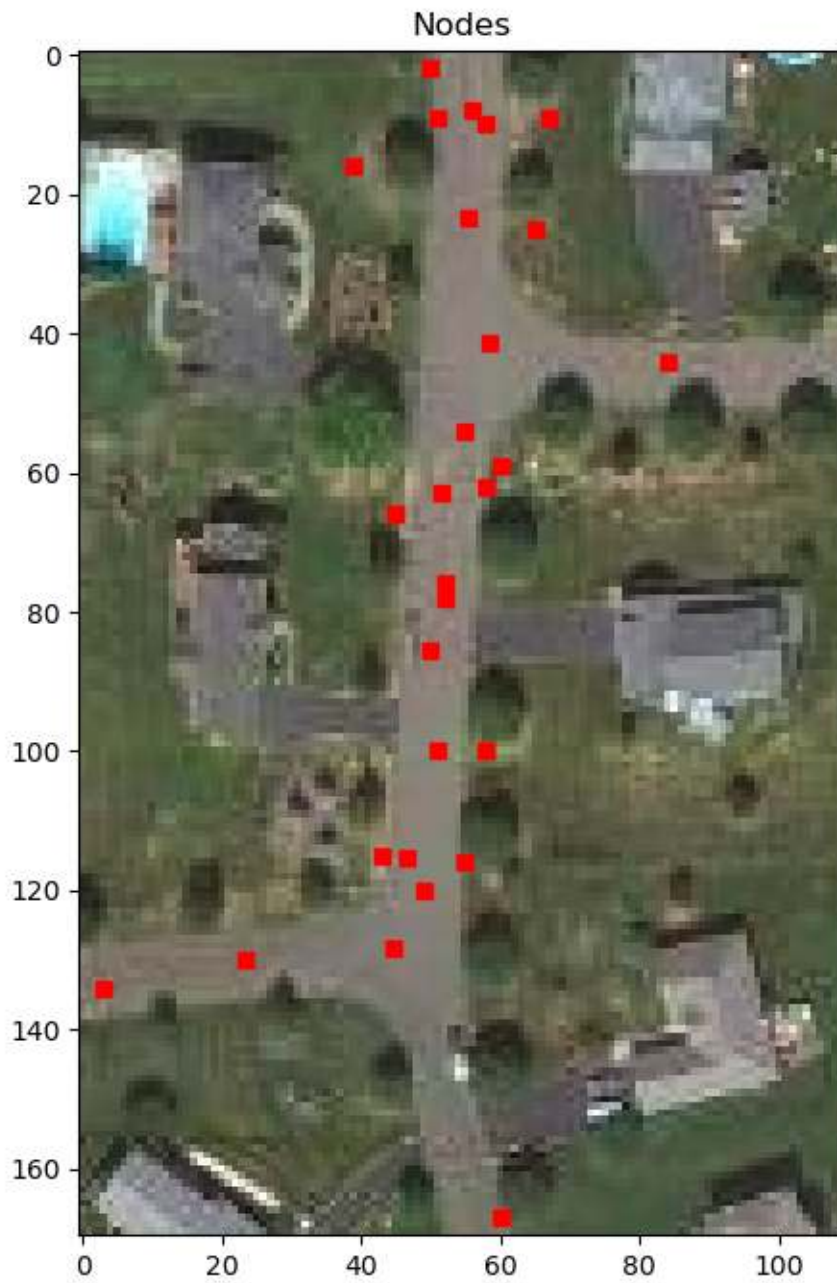
node_dict = {}
for c_node in np.unique(node_id_image[node_id_image > 0]):
    y_n, x_n = np.where(node_id_image == c_node)
```

(continues on next page)



(continued from previous page)

```
node_dict[c_node] = {'x': np.mean(x_n),  
                    'y': np.mean(y_n),  
                    'width': np.mean(dist_map[node_id_image == c_node])}
```



### Analyze edges and connections

We now want to analyze the network by connecting nodes with edges and create a dictionary with more information about the connected segments.

The connection is done in the same way as the toy example with two node and two edges above, i.e.:

1. Dilate each edge id the edge image with a 3x3 structure element
2. Find the nodes that are covered by the dilated edges (there should be only two)
3. Create a dictionary with measured information like
  1. connected nodes
  2. length in pixels
  3. Euclidean distance between the end points
  4. Max width of the edge
  5. Average width of the edge
4. Compute the edge matrix that tells which nodes are connected and how many pixels the segment contains, i.e. the length.

the widths are given by the distance map value of the original structure at the skeleton line.

```
edge_dict = {}
edge_matrix = np.eye(len(node_dict)+1)

s_nodes = []
e_nodes = []
for c_edge in np.unique(edge_id_image[edge_id_image > 0]):
    edge_grow_mask = dilation(edge_id_image == c_edge, np.ones((3, 3)))
    v_nodes = np.unique(node_id_image[edge_grow_mask > 0])
    v_nodes = [v for v in v_nodes if v > 0]
    if c_edge < 6:
        print('Edge', c_edge, 'connects', v_nodes)
    if len(v_nodes) == 2:
        edge_dict[c_edge] = {'start': v_nodes[0],
                             'end': v_nodes[-1],
                             'length': np.sum(edge_id_image == c_edge),
                             'euclidean_distance': np.sqrt(np.square(node_dict[v_
nodes[0]]['x'] -
node_dict[v_
nodes[-1]]['x']) +
np.square(node_dict[v_
nodes[0]]['y'] -
node_dict[v_
nodes[-1]]['y'])
),
                             'max_width': np.max(dist_map[edge_id_image == c_edge]),
                             'mean_width': np.mean(dist_map[edge_id_image == c_edge])}
        edge_matrix[v_nodes[0], v_nodes[-1]] = np.sum(edge_id_image == c_edge)
        edge_matrix[v_nodes[-1], v_nodes[0]] = np.sum(edge_id_image == c_edge)
        s_nodes.append(node_dict[v_nodes[0]])
        e_nodes.append(node_dict[v_nodes[-1]])
```

```
Edge 1 connects [np.int32(1), np.int32(2)]
Edge 2 connects [np.int32(2), np.int32(3)]
```

(continues on next page)

(continued from previous page)

```
Edge 3 connects [np.int32(2), np.int32(5)]
Edge 4 connects [np.int32(4), np.int32(5)]
Edge 5 connects [np.int32(5), np.int32(7)]
```

## Looking at the segment measurements

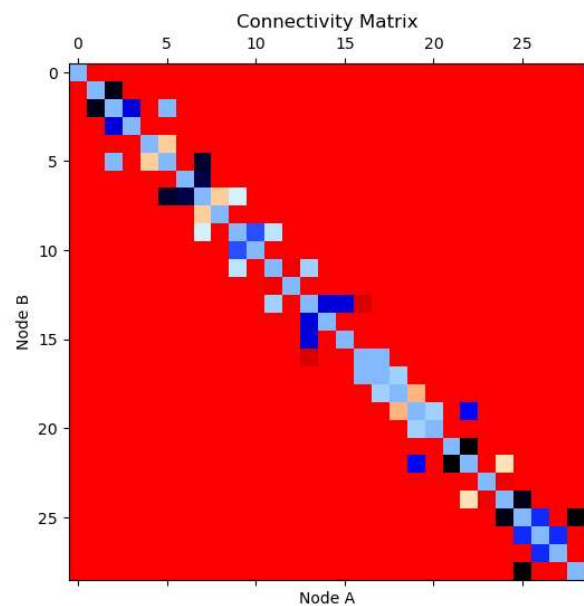
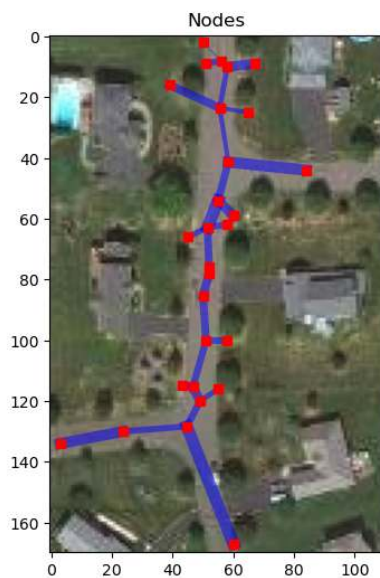
```
skel_df = pd.DataFrame(edge_dict).transpose()
skel_df.sample(10)
```

|    | start | end  | length | euclidean_distance | max_width | mean_width |
|----|-------|------|--------|--------------------|-----------|------------|
| 30 | 26.0  | 27.0 | 19.0   | 20.886599          | 5.385165  | 4.713089   |
| 4  | 4.0   | 5.0  | 8.0    | 9.055385           | 5.656854  | 3.420292   |
| 10 | 9.0   | 11.0 | 11.0   | 13.097922          | 9.433981  | 7.948945   |
| 19 | 17.0  | 18.0 | 6.0    | 7.762087           | 5.385165  | 4.517252   |
| 11 | 11.0  | 13.0 | 6.0    | 9.501462           | 7.071068  | 6.264952   |
| 1  | 1.0   | 2.0  | 7.0    | 8.485281           | 5.830952  | 3.992793   |
| 31 | 25.0  | 28.0 | 40.0   | 41.595940          | 8.602325  | 4.513106   |
| 8  | 7.0   | 9.0  | 16.0   | 18.057008          | 9.848858  | 7.229723   |
| 17 | 16.0  | 17.0 | 1.0    | 2.000000           | 5.656854  | 5.656854   |
| 5  | 5.0   | 7.0  | 12.0   | 13.729530          | 6.403124  | 5.478938   |

## Display the analysis

Here we show the network with the node connected by the edges. The edge line width is controlled by the segment with from the analysis.

The connectivity matrix shows that the node labels are relatively local in this case. The exception is at the branches where there the node numbering go a bit apart.



## 0.10 Skeleton analysis - Tortuosity

One of the more interesting ones in material science is called tortuosity and it is defined as the ratio between the arc-length of a *segment* and the distance between its starting and ending points.

$$\tau = \frac{C}{L}$$

- L - Distance between start and end point
- C - Length of the curve segment

### 0.10.1 Interpreting tortuosity

A high degree of **tortuosity** indicates that the network is convoluted and is important when estimating or predicting flow rates.

Specifically

- in geology it is an indication that diffusion and fluid transport will occur more slowly
- in analytical chemistry it is utilized to perform size exclusion chromatography
- in vascular tissue it can be a sign of pathology.

Low tortuosity



High tortuosity



### 0.10.2 Tortuosity of the street

We see that the tortuosity is relatively low in the street example. This is because most elements are rather straight here.

The opacity is controlled in the network shown in the image to the right. This shows that most elements are opaque, the transparent segments are relatively short. From this observation we can draw the conclusion that the tortuosity is less precise for short segments. This makes sense as we can only use the discrete steps given by the pixels.

```
fig, (ax0,ax1) = plt.subplots(1, 2, figsize=(15, 5))
ax1.imshow(im_crop)
t=[]
for _, d_values in edge_dict.items():
    v_nodes = [d_values['start'], d_values['end']]
    t = t + [(d_values['length'])/d_values['euclidean_distance']]
    s_node = node_dict[v_nodes[0]]
    e_node = node_dict[v_nodes[-1]]
    tt=t[-1]
```

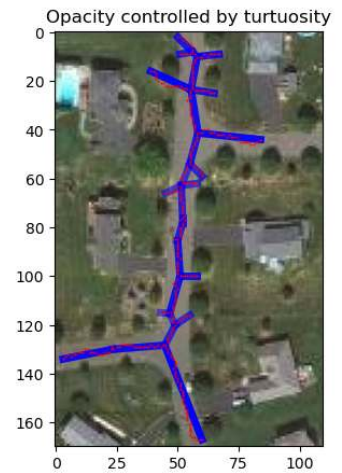
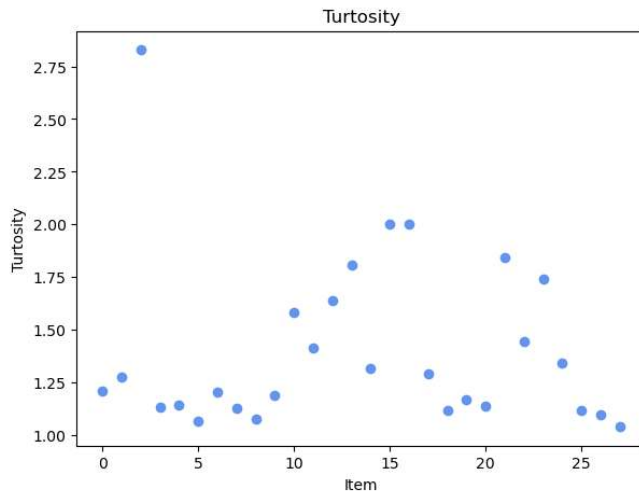
(continues on next page)

(continued from previous page)

```

if tt>1 :
    tt=1.0
ax1.plot([s_node['x'], e_node['x']],
         [s_node['y'], e_node['y']], 'b-',
         linewidth=5, alpha=tt)
ax0.plot(1.0/np.array(t), 'o', color='cornflowerblue');
ax0.set(title='Turtosity', xlabel='Item', ylabel='Turtosity');
ax1.set_title('Opacity controlled by turtuosity')
x,y = np.where(width_skeleton)
ax1.plot(y,x,'.',markersize=1, color='red');

```



### 0.10.3 Further ways to visualize the network

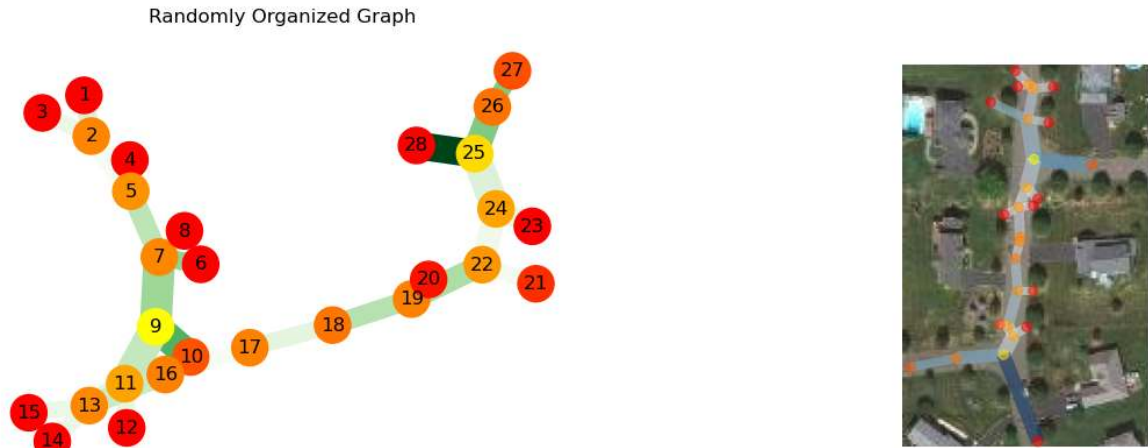
1. Randomly organized graph
  - Nodes colored by the width
  - Edges colored by the length and width set by segment width
2. Add width and length information in the original picture

```

import networkx as nx

G = nx.Graph()
for k, v in node_dict.items():
    G.add_node(k, weight=v['width'])
for k, v in edge_dict.items():
    G.add_edge(v['start'], v['end'], **v)

```



### 0.10.4 Graph Analysis

Once the data has been represented in a graph form, we can begin to analyze some of graph aspects of it, like the degree and connectivity plots.

In graph theory, the degree of a vertex in a graph is the number of edges incident to that vertex. For directed graphs, the degree can be further categorized into the in-degree (number of edges coming into the vertex) and out-degree (number of edges leaving the vertex).

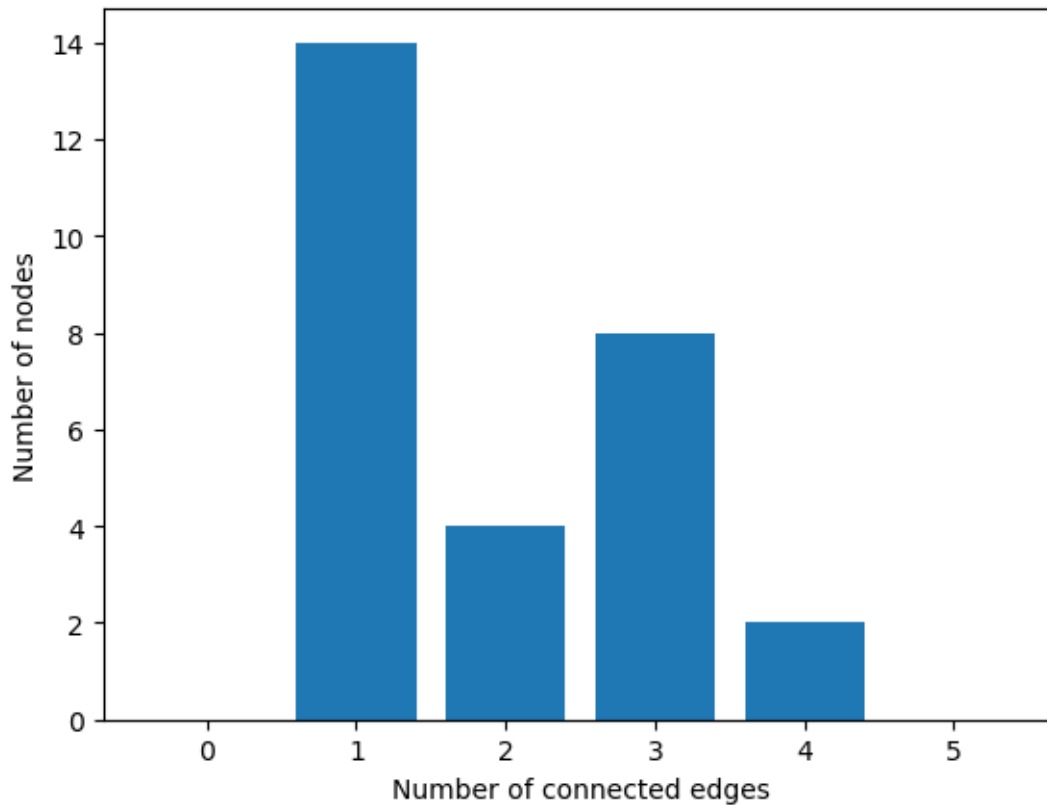
The degree of a graph is the maximum degree among all its vertices. It provides insights into the connectivity of the graph and is often used to classify graphs into different types, such as regular graphs (where all vertices have the same degree) or irregular graphs (where vertices have varying degrees).

Additionally, the sum of degrees of all vertices in a graph is twice the number of edges, which is a consequence of the Handshaking Lemma in graph theory. This property is often useful in various graph-theoretical proofs and calculations.

```
degree_sequence = sorted([d for n, d in G.degree()],
                          reverse=True) # degree sequence

hb, ha = np.histogram(degree_sequence, bins=np.arange(0, 7))

plt.bar(ha[:-1], hb, align="center", plt.xlabel('Number of connected edges'), plt.
        ylabel('Number of nodes');
```



### 0.10.5 Skeletons going 3D

- Object topology must still be preserved
- More complex neighborhoods to analyze

[A skeletonization algorithm survey](#)

### Application of 3D skeletons: Root networks

#### Motivation

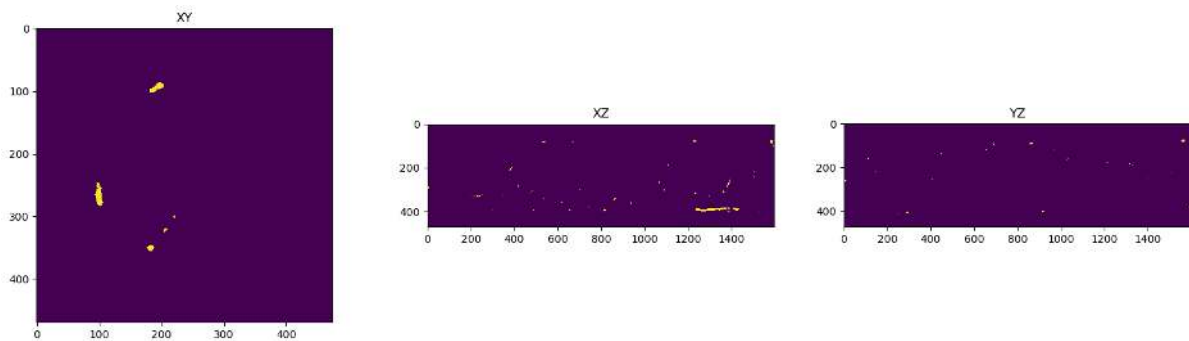
#### Why analyzing root networks

- Soil stability
- Water uptake
- Growth under different conditions

### Way to analyze the network

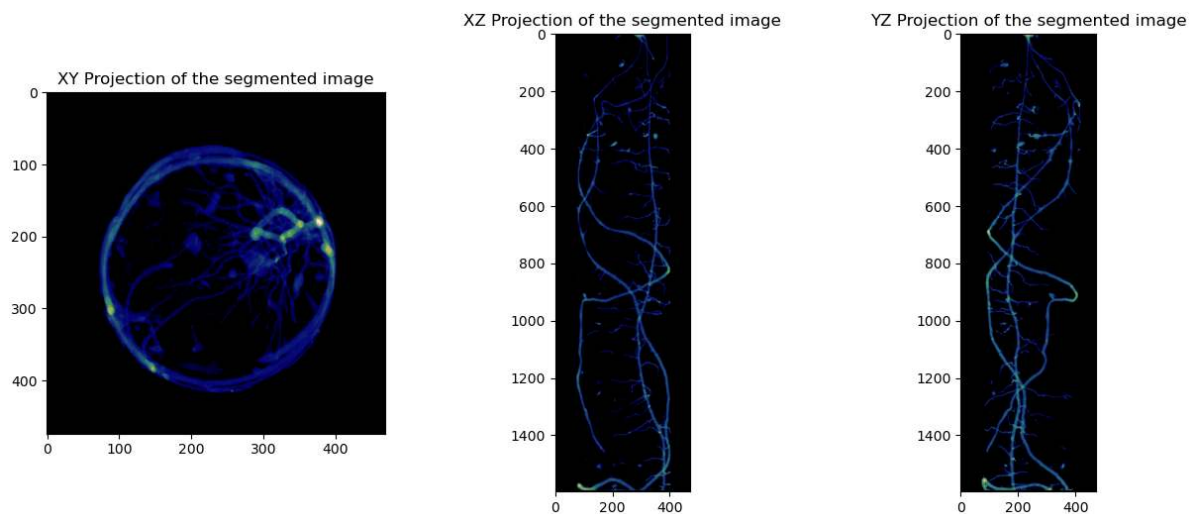
- Root network morphology
  - Segment length
  - Branching
  - Branch orientation
  - Tortuosity
- Influence volumes

### Load volume with roots



### Let's look at the projection instead

The slice planes did not reveal the structure of the volume very well. Here is an alternative. Sum all pixels along one axis to obtain a 2D projection of the structure. This is a useful approach for sparse structures.

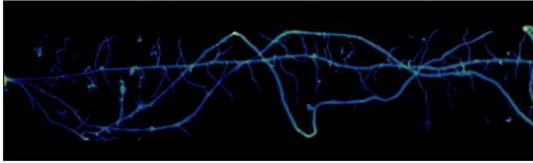




## Create the 3D skeleton

```
skel = skeletonize(root)
```

Sum-projection of the segmented image



Max-projection of the skeleton



## Detailed 3D view of skeleton

## 0.11 Segmenting touching items

Watershed is a method for segmenting objects without using component labeling.

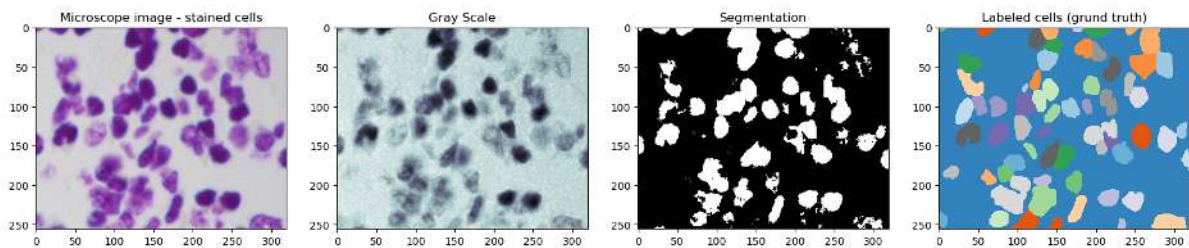
- It utilizes the shape of structures to find objects
- From the distance map we can make out substructures with our eyes
- But how to we find them?!

### 0.11.1 Watershed

We use a sample image now from the [Datascience Bowl 2018 from Kaggle](#). The challenge is to identify nuclei in histology images to eventually find cancer better. The winner tweeted about the solution [here](#)

### 0.11.2 Let's load a cell image

The test image for this part is a microscope image of stained cells. The original is a color image, but we will convert it to grayscale for the segmentation. The thresholded image has quite many misclassifications as you can see in the provided ground truth images to the very right.



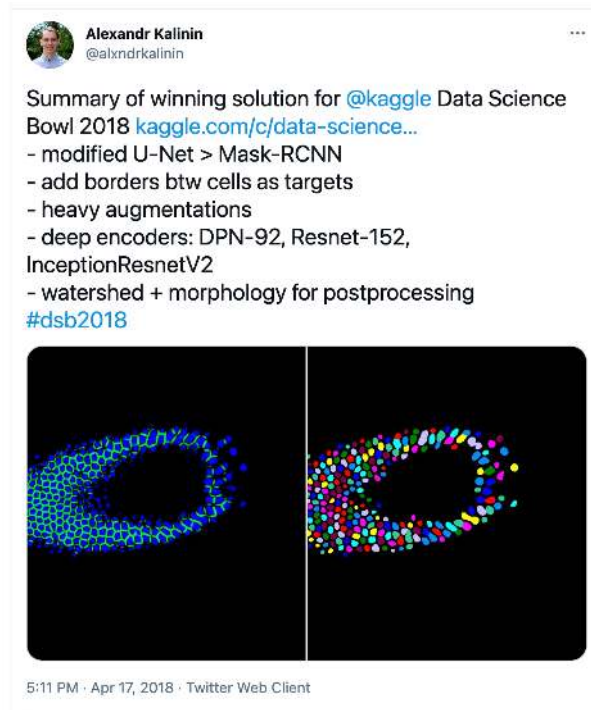


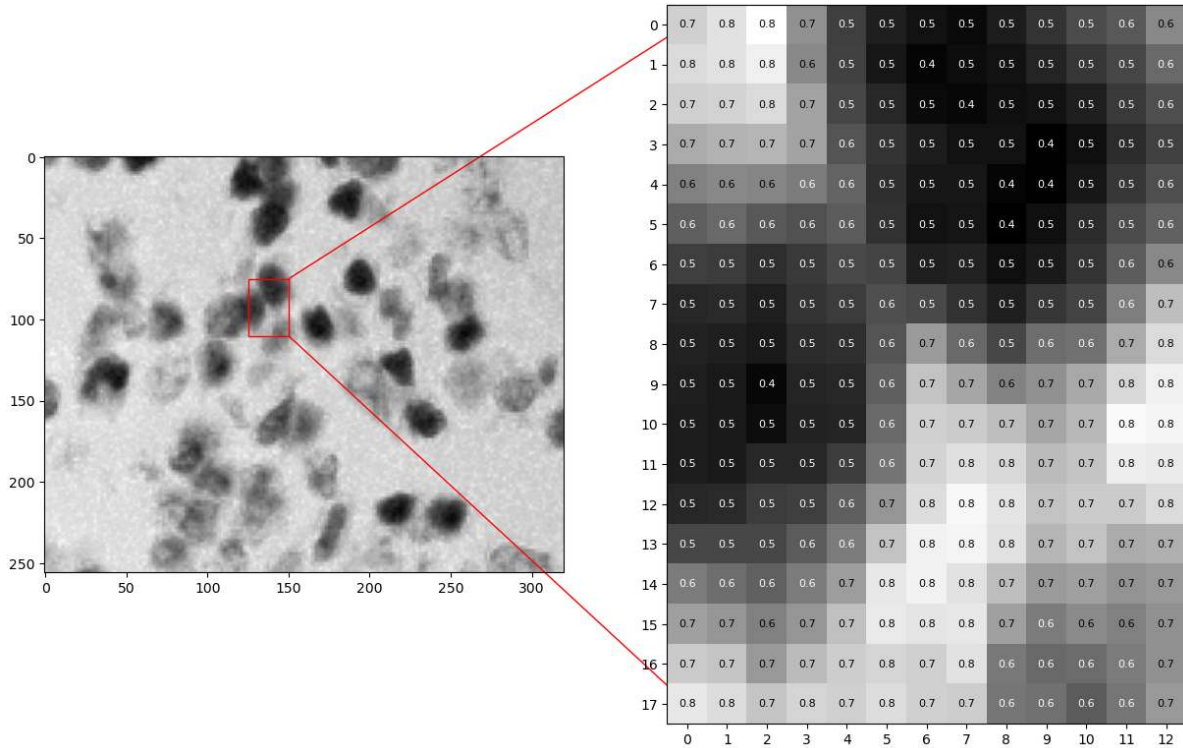
Fig. 1: Tweet by Alexandr Kalinin about the post processing after ML-based segmentation

### 0.11.3 Try component labeling

#### First reduce the data

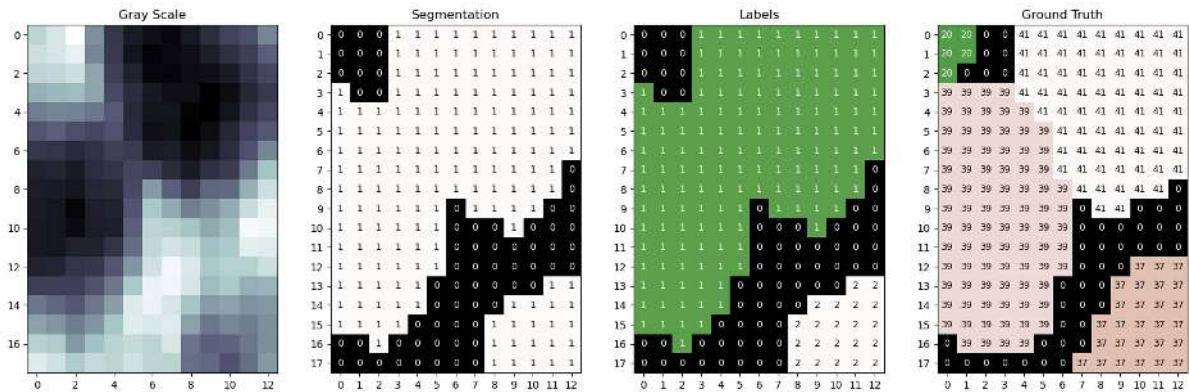
The full image has too much detail. We crop the image to only contain few cells.

```
from skimage.morphology import label
bw_roi = bw_img[75:110:2, 125:150:2]
```



### Trying component labelling on the cells

```
bw_roi_seg = bw_roi < threshold_otsu(bw_img)
bw_roi_label = label(bw_roi_seg)
```



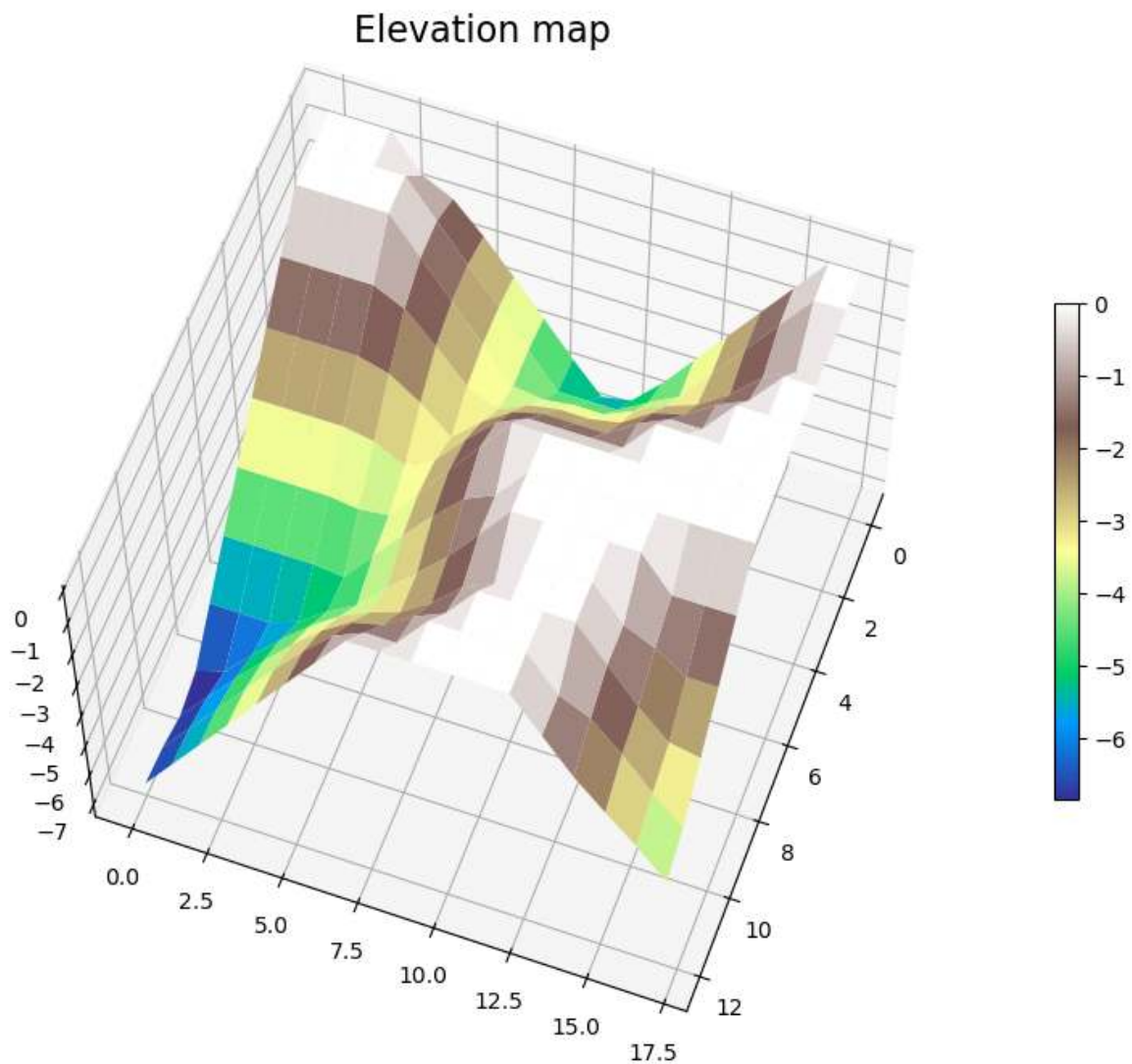
When we compare the ground truth with the connected component labeling, we see that it fails to identify the cells. This demonstrates the weakness of the labeling algorithm - it is not able to label items that touch. We need something else to handle this use case.

## 0.12 Watershed: Flowing Downhill

Watershed segmentation is a well-known labeling method for complicated scenes where the items are connected or overlapping. The name comes from the analogy in nature; if you pour a bucket of water in a hilly landscape, it will flow downhill and at the end reach the bottom of the valley. If there is a thunderstorm the water will form small lakes in the landscape, these lakes will continue to grow until they reach the ridge of the hill where they will spill over into the next valley.

The watershed algorithm works in the same manner, where the lakes are the segmented items. However, in the case of segmentation, we imagine building dams at the ridges to prevent spillover. These dams form the item boundaries.

We can imagine watershed as waterflowing down hill into basins. The topology in this case is given by the distance map



### 0.12.1 Preparations for watershed segmentation

The segmented image is a bi-level image which not a good representation of a landscape. Therefore, we need to process the bi-level image with a few steps to provide the information the watershed algorithm requires.

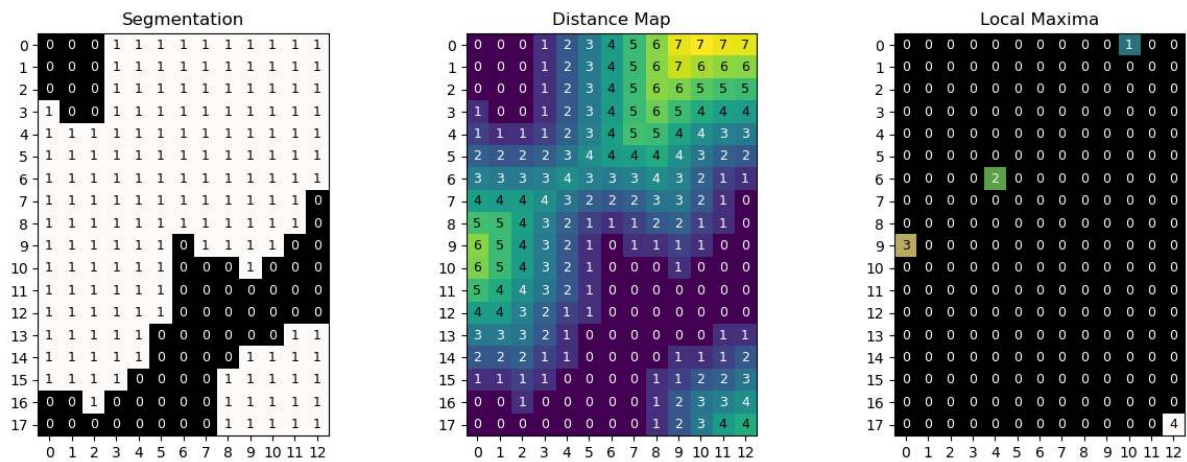
We need to create a seed image

1. Segment image to bi-level
2. Compute distance map (and flip upside down)
3. Identify local maxima

```
from skimage.feature import peak_local_max
from scipy.ndimage import distance_transform_edt

coords = peak_local_max(distance_transform_edt(bw_roi_seg), footprint=np.ones((3, 3)),
                        labels=bw_roi_seg, exclude_border=False)
roi_local_maxi = np.zeros(bw_roi_dmap.shape, dtype=bool)
roi_local_maxi[tuple(coords.T)] = True

# Seed labels
labeled_maxi = label(roi_local_maxi)
```



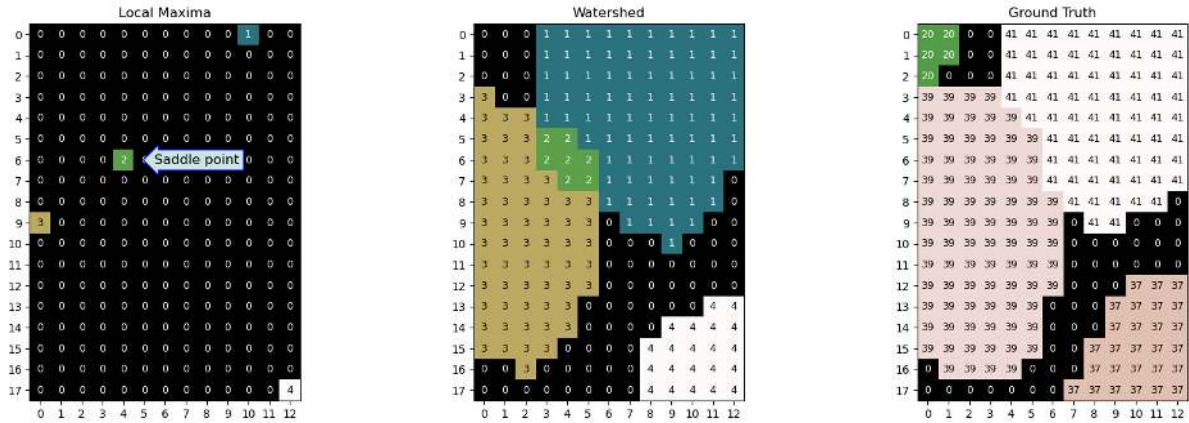
The labeled local maxima pixels will be used to set the label of each 'catchment basin' in the watershed segmented image. The basin growing is guided by the distance map.

### 0.12.2 Run watershed

When we run the watershed on the distance map and local maxima markers we get the following result. Note here that there is a minus sign for the distance map to flip the levels making the greatest distance the deepest point of the valley.

```
from skimage.segmentation import watershed
ws_labels = watershed(-bw_roi_dmap, labeled_maxi, mask=bw_roi_seg)
```





The segmentation was only partially successful when we compare to the ground truth.

1. A segment in the upper left corner is missing.
2. The item shapes are not similar to the ground truth.
3. There is a small object (label 2) between the large two objects (labels 1 and 3).

There are several reasons for these observations

1. The first two are issues are related to the thresholding accuracy. A different segmentation algorithm would probably be needed to address these issues.
2. The small item between the larger ones is produced as an artifact of the peak finder which also identifies saddle points.

## 0.12.3 Removing too small elements - Method 1

One of the components (Label=2) is too small!

We can remove it by deleting unwanted seeds.

- seed belonging to the bottom 10 percentile of areas
- rerunning watershed

```
label_area_dict = {i: np.sum(ws_labels == i)
                   for i in np.unique(ws_labels[ws_labels > 0])} # this could also be
done with regprops.
clean_label_maxi = labeled_maxi.copy()
area_cutoff = np.percentile(list(label_area_dict.values()), 10)
print('Cutoff at {0:0.2f}'.format(area_cutoff))
for i, k in label_area_dict.items():
    print('Label: ', i, 'Area:', k, 'Keep:', k > area_cutoff)
    if k <= area_cutoff:
        clean_label_maxi[clean_label_maxi == i] = 0
```

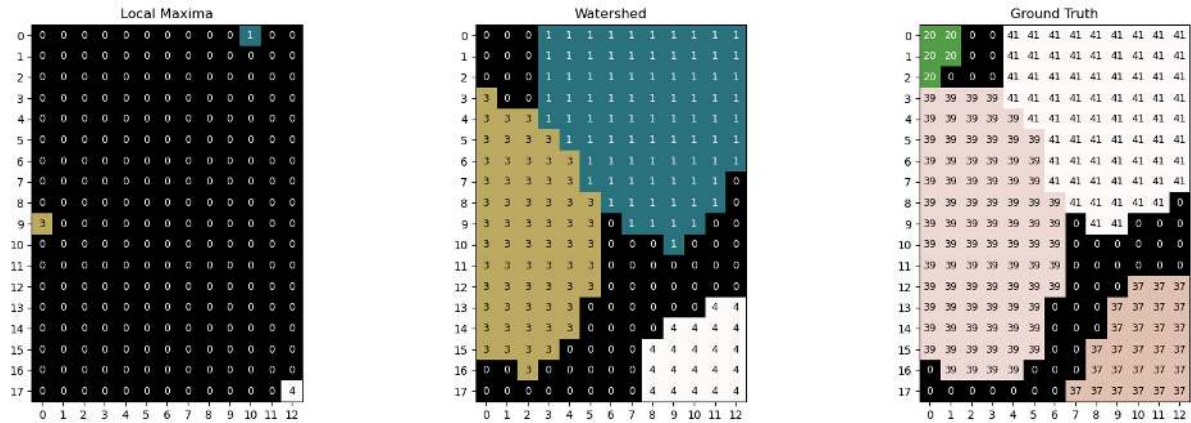
```
Cutoff at 11.20
Label: 1 Area: 82 Keep: True
Label: 2 Area: 7 Keep: False
Label: 3 Area: 59 Keep: True
Label: 4 Area: 21 Keep: True
```

This approach thus requires to watershed runs to guarantee that all objects have the correct size.

## Watershed after removing labels

Now, let's run watershed again to observe the improvement of the cleaning operation.

```
ws_labels = watershed(-bw_roi_dmap, clean_label_maxi, mask=bw_roi_seg)
```



This result looks much more convincing than the component labeling before. We do, however, still fail to segment one region compared to the ground truth. This is because our thresholding didn't manage to mark the weaker intensity cell as foreground. So, to get this one, we need to revise our cell identification segmentation.

### 0.12.4 Scaling back up

Now we can perform the operation on the whole image and see how the results look

This time we repeat the whole workflow for labeling the items in the image.

1. Thresholding with Otsu followed by an opening operation with a disk radius=3 structure element.
2. Compute the distance map.
3. Identify local maxima.
4. Create an image with labeled seeds.
5. First watershed run.
6. Compute item areas.
7. Run a size filter to remove the 20% item with the smallest area.
8. Update the seed image.
9. Second watershed run.

```
bw_seg_img = opening(bw_img < threshold_otsu(bw_img), disk(3))

bw_dmap = distance_transform_edt(bw_seg_img)
bw_peak_coords = peak_local_max(bw_dmap.copy(), footprint=np.ones((3, 3)),
                                labels=bw_seg_img, exclude_border=True)

bw_dmap[bw_dmap<0]=0
bw_peaks = np.zeros_like(bw_dmap)
bw_peaks[tuple(bw_peak_coords.T)]=1
bw_peaks = label(bw_peaks)
ws_labels = watershed(-bw_dmap, bw_peaks, mask=bw_seg_img)
```

(continues on next page)

(continued from previous page)

```
label_area_dict = {i: np.sum(ws_labels == i)
                   for i in np.unique(ws_labels[ws_labels > 0])}

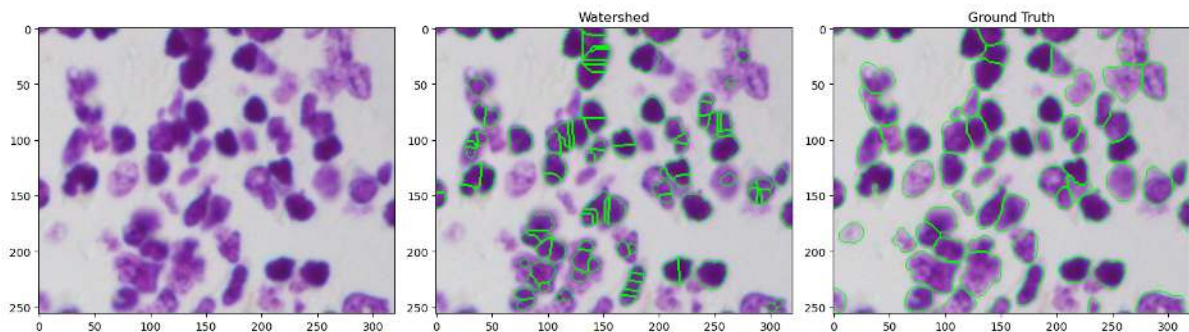
clean_label_maxi = bw_peaks.copy()
lab_areas = list(label_area_dict.values())
area_cutoff = np.percentile(lab_areas, 20)
print('20% cutoff', area_cutoff, '. Removed', np.sum(
    np.array(lab_areas) < area_cutoff), 'components')
for i, k in label_area_dict.items():
    if k <= area_cutoff:
        clean_label_maxi[clean_label_maxi == i] = 0

ws_labels = watershed(-bw_dmap, clean_label_maxi, mask=bw_seg_img)
```

```
20% cutoff 35.0 . Removed 25 components
```

### The resulting segmentation - no fantastic performance

Let's look at the result of the WS segmentation with overlaid on the original image.



When we compare our segmentation to the ground truth, we see that

1. Many cells are missing.
2. The boundaries are too conservative.
3. The labeled cells are often fragmented into several smaller segments.

### Summarizing the result

- many over-segmented cells
- missing cells



## Why bad performance?

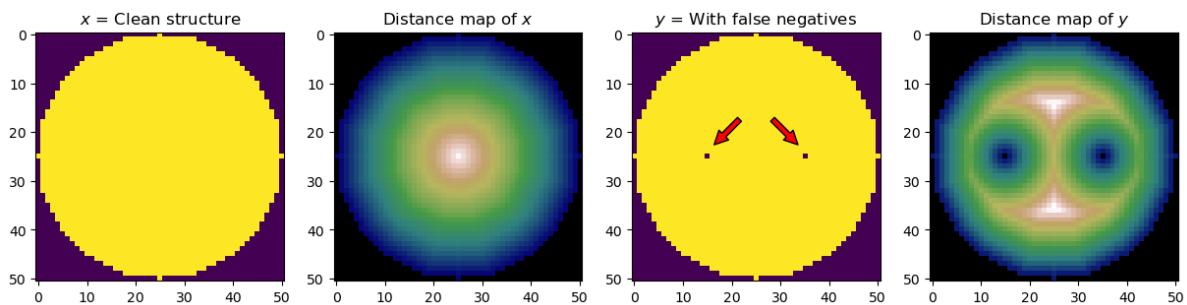
### Missing objects

- Too simple thresholding method was used (Otsu)

### Over segmentation

- Irregular shapes and misclassified pixels produce too many local maxima in the elevation map

The example below illustrates the effect of two misclassified pixels. The distance map of the clean object has a monotonously increasing intensity with the maximum in the center of the circle. When we however inject two pixels to mimic misclassified pixel inside object and compute distance map, we obtain an irregular distance map. The map now has two additional local minima inside the object and also a second local maximum. These new structures in the distance map will guide the watershed algorithm to place a horizontal separation line in the segmented image. Thus, we see the origin of the over-segmentation.



### Solutions

- Find better segmentation approach than only histogram
- Use morphological algorithms like h-max and min-impose [Soille, 1999](#)

## 0.12.5 Removing too small elements - method 2

The over segmentation is a know problem of the watershed segmentation.

A method proposed by [Soille, 1999](#) is to

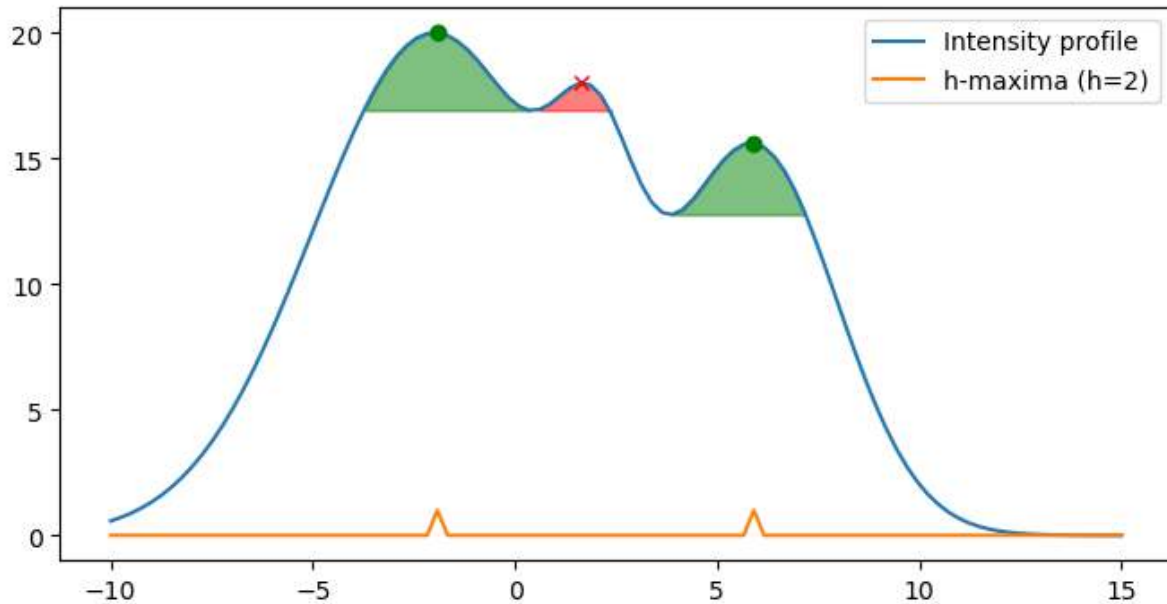
1. Analyze the distance map to reject low amplitude peaks.
2. Strengthen the local minima.

This can be done using methods called reconstruction by dilation  $R_f^\delta(g)$  and erosion  $R_f^\varepsilon(g)$ . These are iterative algorithms that reconstructs a mutual minimum or maximum between a mask image and the reconstructed image.

[Soille, 1999](#)

### Use h-max to find peaks

- The find peaks method finds any peak in the image.
- h-Max add the criterion that the peak must be at least  $h$  greylevels higher than the surroundings to produce a marker.



The h-max finds all peaks with a local height of  $h$ . The green peaks in the curve above will be kept.

The advantage of using the h-max over the previous approach to remove small objects is two-fold.

1. A second watershed run is not needed - The small items have already been removed in the peak identification process.
2. Some over-segmentation can be reduced.

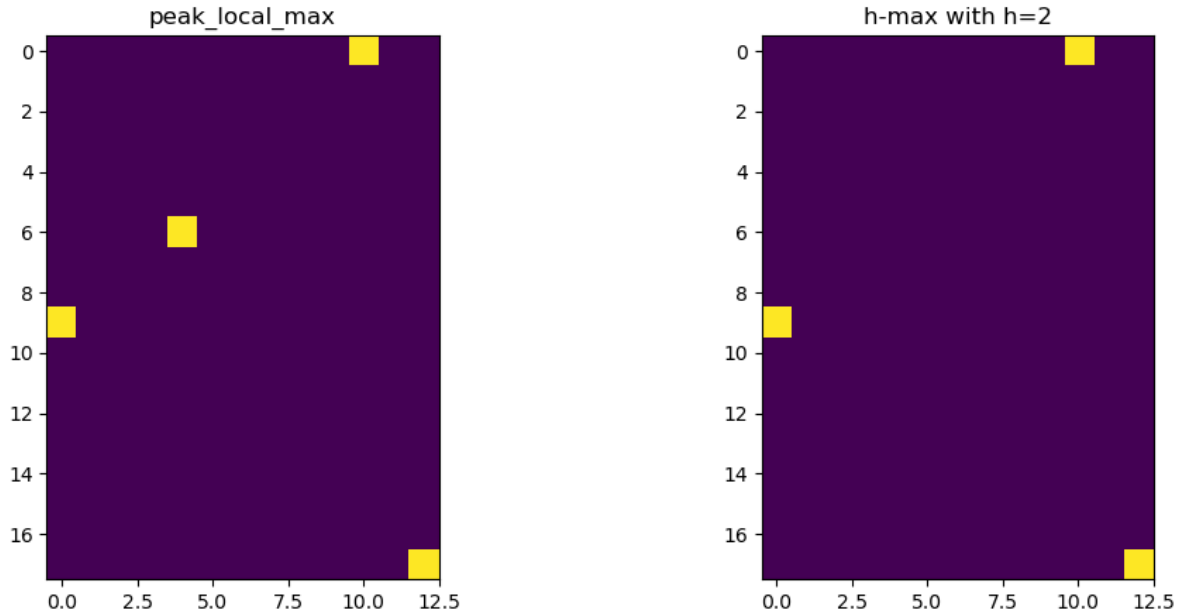
### Applying h-max

Let's try the h-max to produce the seed mask on the small test image. Here we will use  $h=2$ .

```
from skimage.morphology import h_maxima

roi_local_maxi = peak_local_max(bw_roi_dmap.copy(), footprint=np.ones((3, 3)),
                                labels=bw_roi_seg, exclude_border=False)
bw_roi_dmap[bw_roi_dmap<0]=0
bw_peaks = np.zeros_like(bw_roi_dmap)
bw_peaks[tuple(roi_local_maxi.T)]=1

h=2
hmax=h_maxima(bw_roi_dmap,h=h)
```



It seems that h-max was able to exclude the saddle point which was identified by the peak finder approach.

### Use min-impose to strengthen the peaks

$$R_{(f+1)f_m}^{\varepsilon}(f_m)$$

with

$$f_m(p) = \begin{cases} 0 & \text{if } p \text{ belongs to a marker} \\ t_{max} & \text{otherwise} \end{cases}$$

*Compare to pulling a wrinkled membrane.*

```
from skimage.morphology import grayreconstruct as gr

def min_impose(dimg, markers) :
    fm=markers.copy()
    fm[markers != 0] = 0
    fm[markers == 0] = dimg.max()
    dimg2 = np.minimum(fm, dimg+1)
    res = gr.reconstruction(fm, dimg2, method='erosion')

    return res
```

### Testing minimpose and h-min - cropped image

Equipped with these new tools we compare the WS result with the first approach.

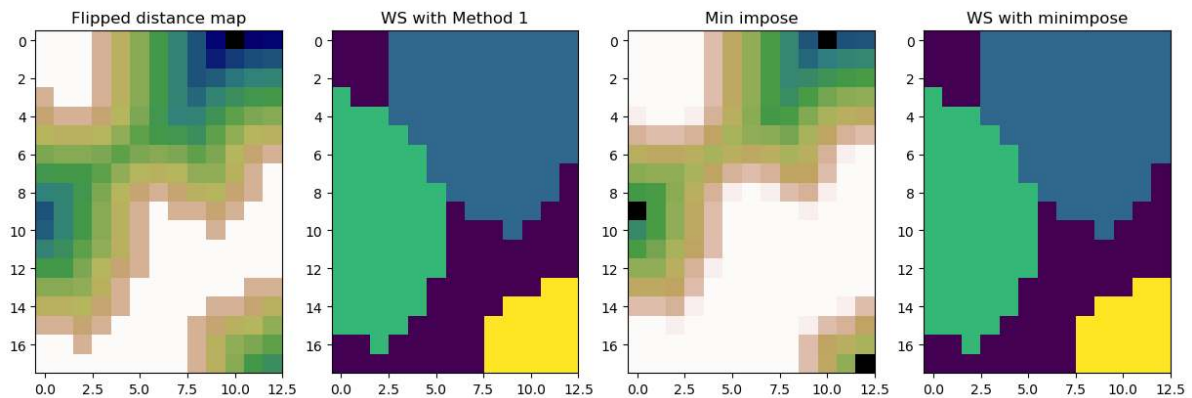
```
d = bw_roi_dmap.max()-bw_roi_dmap

ws0 = watershed(-bw_roi_dmap, label(hmax), mask=bw_roi_seg)
m3=label(hmax)
minimp_h=min_impose(d, hmax)
```

(continues on next page)

(continued from previous page)

```
ws = watershed(minimp_h, m3, mask=bw_roi_seg)
```



... not much difference here!

There is no change in the final WS segmented images at this level. You however see a difference in the distance maps from the two approaches.

The min-imposed distance map is smoother than the Euclidean distance map. This has a regularizing effect on the WS algorithm and produces more reliable structures.

### Testing minimpose and h-min - full image

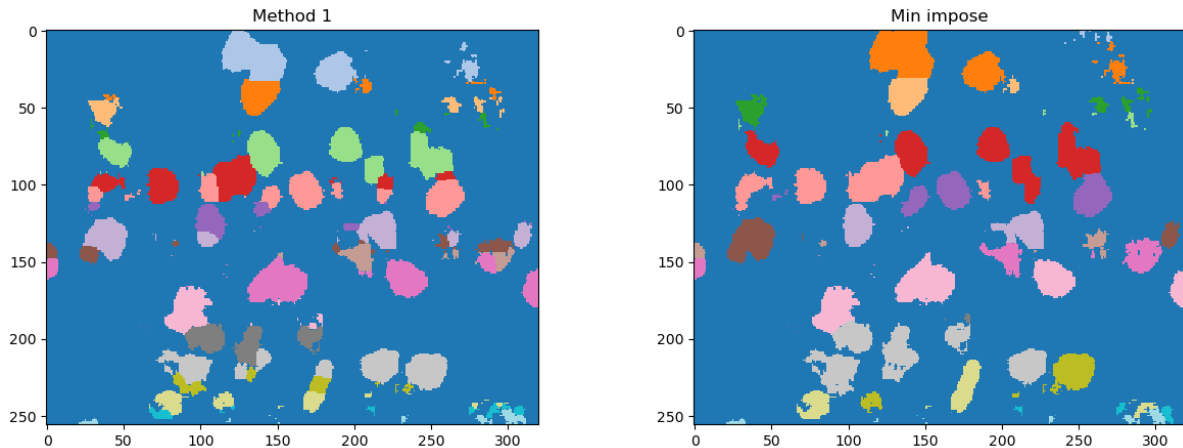
Let's compare the two approaches on the full image using the same workflows as before.

```
seg=bw_img < threshold_otsu(bw_img)
dmap = distance_transform_edt(seg)

peak_idx = peak_local_max(dmap, footprint=np.ones((3, 3)), labels=seg, exclude_
    <del>border=False</del>)
peak_mask = np.zeros_like(dmap, dtype=bool)
peak_mask[tuple(peak_idx.T)] = True
m0 = label(peak_mask)

ws0 = watershed(-dmap, m0, mask=seg)

h=1
localmax = h_maxima(dmap,h)
rdmap = dmap.max()-dmap
marker = label(localmax)
ws1 = watershed(min_impove(rdmap,marker),marker,mask=seg)
```

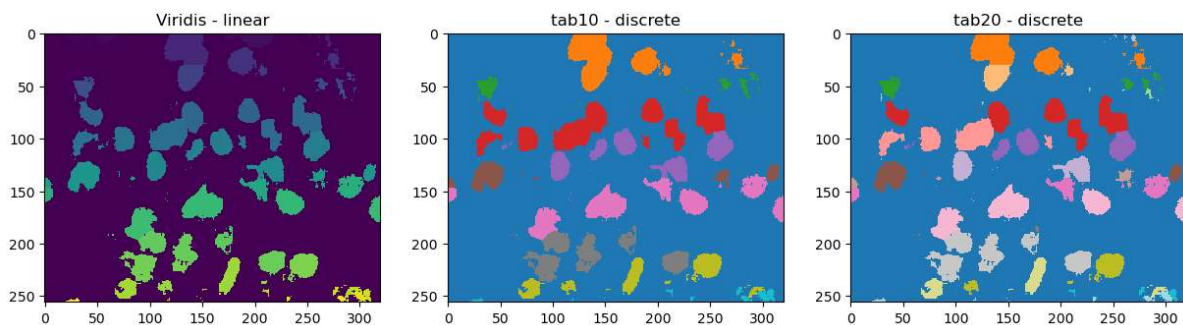


At first sight it looks like both methods have labeled several items with the same label. Even though they are not even connected. Something is wrong here!

## 0.13 Color maps for many labels

### 0.13.1 Standard color maps

It is hard to identify all items in a watershed segmented image



- Linear colormaps like viridis appear like a gradient.
- Discrete colormaps mostly have too few categories, multiple item have the same color.

### 0.13.2 Custom colormaps

A solution to mitigate the visualization problem we have with uniquely visualize many items with too few and similar colors is to

Create custom colormaps with the same number of entries as labels

**Random colors** Randomize the RGB channels and orthogonalize the vectors

```
from scipy.stats import ortho_group
from matplotlib.colors import ListedColormap

def randomCM(N, low=0.2, high=1.0, seed=42, bg=0) :
    np.random.seed(seed=seed)
```

(continues on next page)

(continued from previous page)

```
clist=np.random.uniform(low=low,high=high,size=[N,3]);
m = ortho_group.rvs(dim=3)
if bg is not None : clist[0,:]=bg;

rmap = ListedColormap(clist)

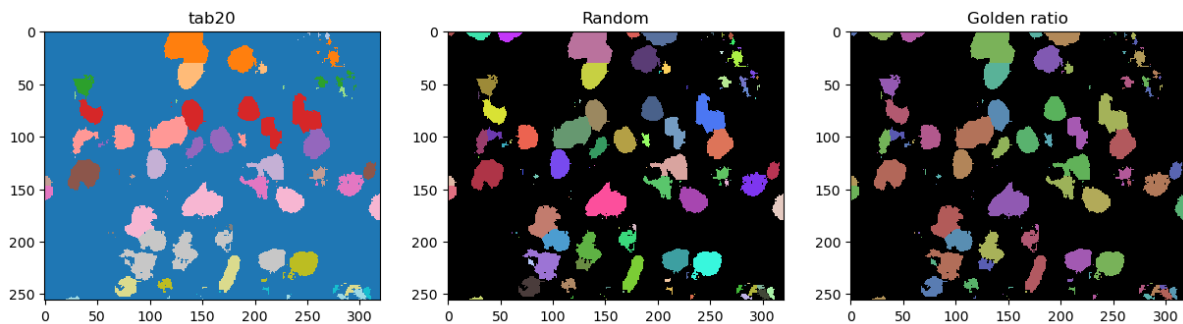
return rmap
```

**Golden ratio** Use the golden ratio to determine the next hue value

```
def goldenCM(N,increment=1.0,s=0.5,v=0.7,bg=0) :
    phi= 0.5*(np.sqrt(5)-1)

    hsv = np.zeros([N,3]);
    hsv[:, 0] = increment*phi*np.linspace(0,N-1,N)-np.floor(increment*phi*np.
    linspace(0,N-1,N))
    hsv[:, 1] = s
    hsv[:, 2] = v
    rgb = hsv2rgb(hsv)
    if bg is not None : rgb[0,:]=bg
    cm = ListedColormap(rgb)
    return cm
```

**Trying the custom colormaps**



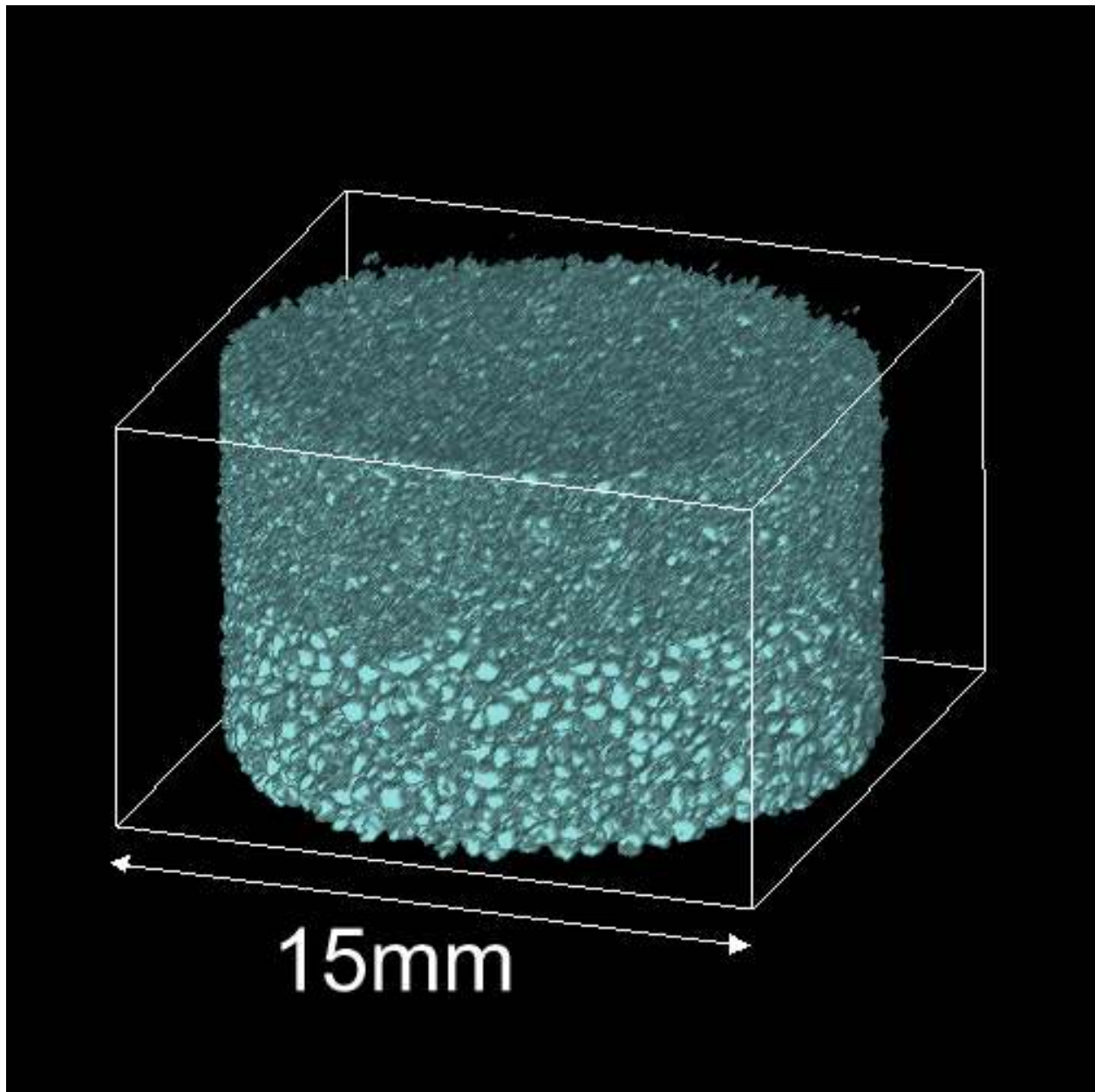
Both colormaps are able to improve the distinction between adjacent items and now we can see that the second method to prepare mask and distance map produces much more reliable items.

## 0.14 Example - Separating grain packings

It is very common to scan grain packings in different fields of materials science.

In this example,

- we have a volume with grains from two size fractions (500x500x300 voxels).



- The purpose of this packing is to study the water flow properties at the interface between the fractions.

**Our task**

- We want to separate the two fractions

Freely from [Kaestner et al. 2005](#)



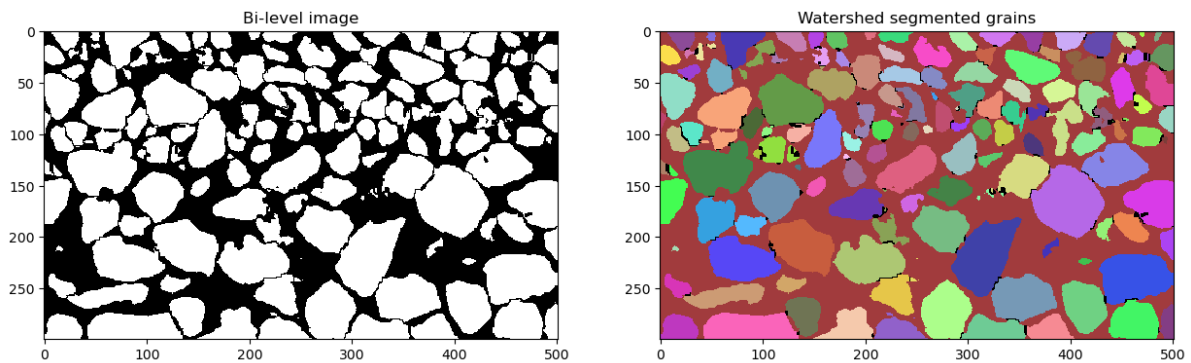
### 0.14.1 Workflow to separate grain packings

0. Load the data
1. Image preparation
  - Noise reduction
  - Binarization
  - Clean misclassified voxels
2. Label the grains
  - Compute distance map
  - Identify and label grain peaks
  - Use watershed segmentation
3. Categorize the grains
  - Compute region information
  - Remove tiny and huge regions
  - Inspect the region properties
  - k-means clustering
4. Assign fraction class to each grain

### 0.14.2 Load and inspect the data

We cheat here and skip the segmentation steps (1 and 2)... (the data is zipped)

```
bi = load_with_unzip('data/grains.npy')  
ws = load_with_unzip('data/ws_grains.npy')
```



These are vertical slices of the bi-level and WS segmented volumes. You can probably see that there is a change in grain size at about the height of 120. This is the location we want to find more precisely using grain size analysis. So, we are ready to start the analysis.

### 0.14.3 Compute region properties

Our first step is to reduce the amount of data to work with; we don't need the full image during most of the analysis. So, let's compute some region properties.

We need:

- The item label - be able to assign the identified class to the final volume.
- Object area (number of voxels in region, same as the volume in this case) - needed to make a size decision
- Object centroid - classification can be guided by the position in addition to the size. It is less likely to find a large object far away from the separation line and otherwise.

```
from skimage.measure import regionprops_table
import pandas as pd
import seaborn as sns

rp = pd.DataFrame.from_dict(regionprops_table(ws, properties=['label', 'area',
↪ 'centroid']))
rp.sample(5)
```

|      | label | area    | centroid-0 | centroid-1 | centroid-2 |
|------|-------|---------|------------|------------|------------|
| 1378 | 1379  | 780.0   | 74.916667  | 1.607692   | 179.276923 |
| 845  | 846   | 33108.0 | 12.254319  | 124.889996 | 81.973783  |
| 422  | 423   | 11271.0 | 288.379292 | 165.538816 | 39.873303  |
| 997  | 998   | 7883.0  | 244.648738 | 216.303945 | 100.990993 |
| 667  | 668   | 12217.0 | 329.069166 | 44.555128  | 57.523124  |

This is a great reduction in amount of data

|   |                 |                |           |              |           |   |
|---|-----------------|----------------|-----------|--------------|-----------|---|
|   | Tomography      | Reconstruction | Filtering | Thresholding | Labeling  | \ |
| 0 | 250000000       | 300000000      | 300000000 | 75000000     | 150000000 |   |
|   | Region analysis |                |           |              |           |   |
| 0 | 6720            |                |           |              |           |   |

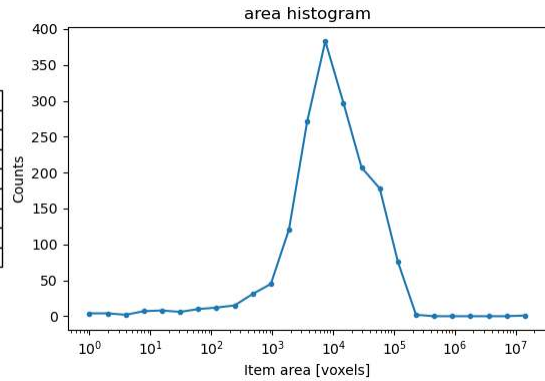
- The image volume was 500x500x300 voxels
- The table has 1680 rows and 5 columns

#### Inspect the table

We can get a quick overview of the item statistics using the function `rp.describe()`. The area is of particular interest here, therefore we plot the histogram. Note, that the item area is on a logarithmic scale to be able to visualize the spread of values.

Summary statistics

|       | label   | area       | centroid-0 | centroid-1 | centroid-2 |
|-------|---------|------------|------------|------------|------------|
| count | 1680.0  | 1680.0     | 1680.0     | 1680.0     | 1680.0     |
| mean  | 878.51  | 44356.23   | 251.0      | 239.14     | 99.05      |
| std   | 1651.14 | 675304.26  | 150.42     | 153.93     | 81.36      |
| min   | 1.0     | 1.0        | 0.76       | 0.0        | 0.0        |
| 25%   | 420.75  | 5785.0     | 118.28     | 104.18     | 36.2       |
| 50%   | 840.5   | 12251.5    | 252.92     | 232.28     | 80.27      |
| 75%   | 1260.25 | 32301.25   | 379.74     | 365.65     | 132.45     |
| max   | 65535.0 | 27663915.0 | 500.0      | 500.0      | 299.0      |



We see that some items in the image are either huge or very tiny.

## 0.14.4 Apply size filters on the table

Let's filter the region property table and extract the grains with realistic size:

- The huge item is the pore space

```
rp['fg'] = rp['area'] < 1e7
```

- The small items are grain fractions at the boundary or over segmentations

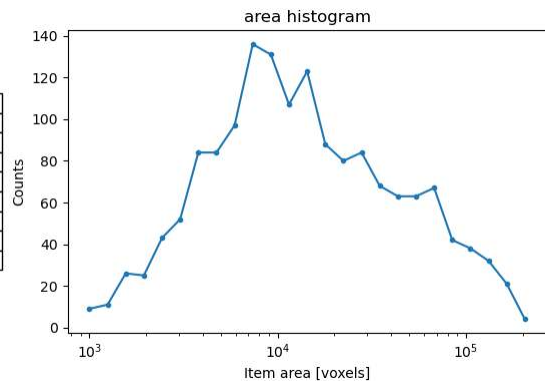
```
rp['small'] = rp['area'] < 1000
```

Finally, we make a new table with only realistic grains:

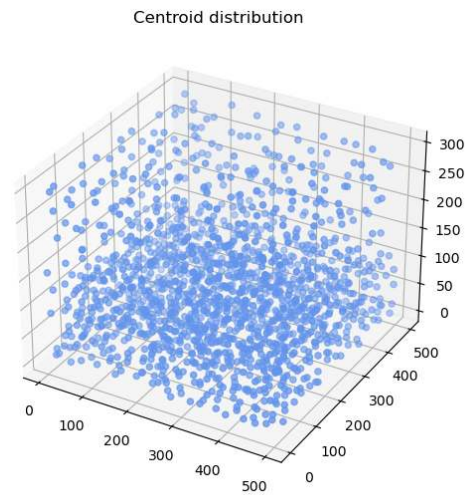
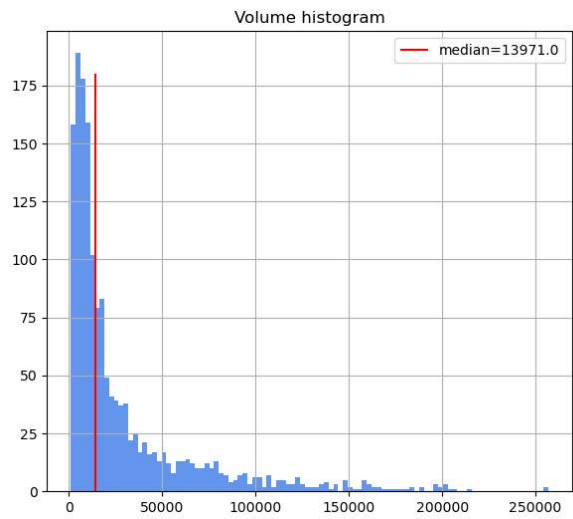
```
grains = rp[(rp['fg'] == True) & (rp['small'] == False)].copy()
```

Summary statistics

|       | label   | area       | centroid-0 | centroid-1 | centroid-2 |
|-------|---------|------------|------------|------------|------------|
| count | 1680.0  | 1680.0     | 1680.0     | 1680.0     | 1680.0     |
| mean  | 878.51  | 44356.23   | 251.0      | 239.14     | 99.05      |
| std   | 1651.14 | 675304.26  | 150.42     | 153.93     | 81.36      |
| min   | 1.0     | 1.0        | 0.76       | 0.0        | 0.0        |
| 25%   | 420.75  | 5785.0     | 118.28     | 104.18     | 36.2       |
| 50%   | 840.5   | 12251.5    | 252.92     | 232.28     | 80.27      |
| 75%   | 1260.25 | 32301.25   | 379.74     | 365.65     | 132.45     |
| max   | 65535.0 | 27663915.0 | 500.0      | 500.0      | 299.0      |

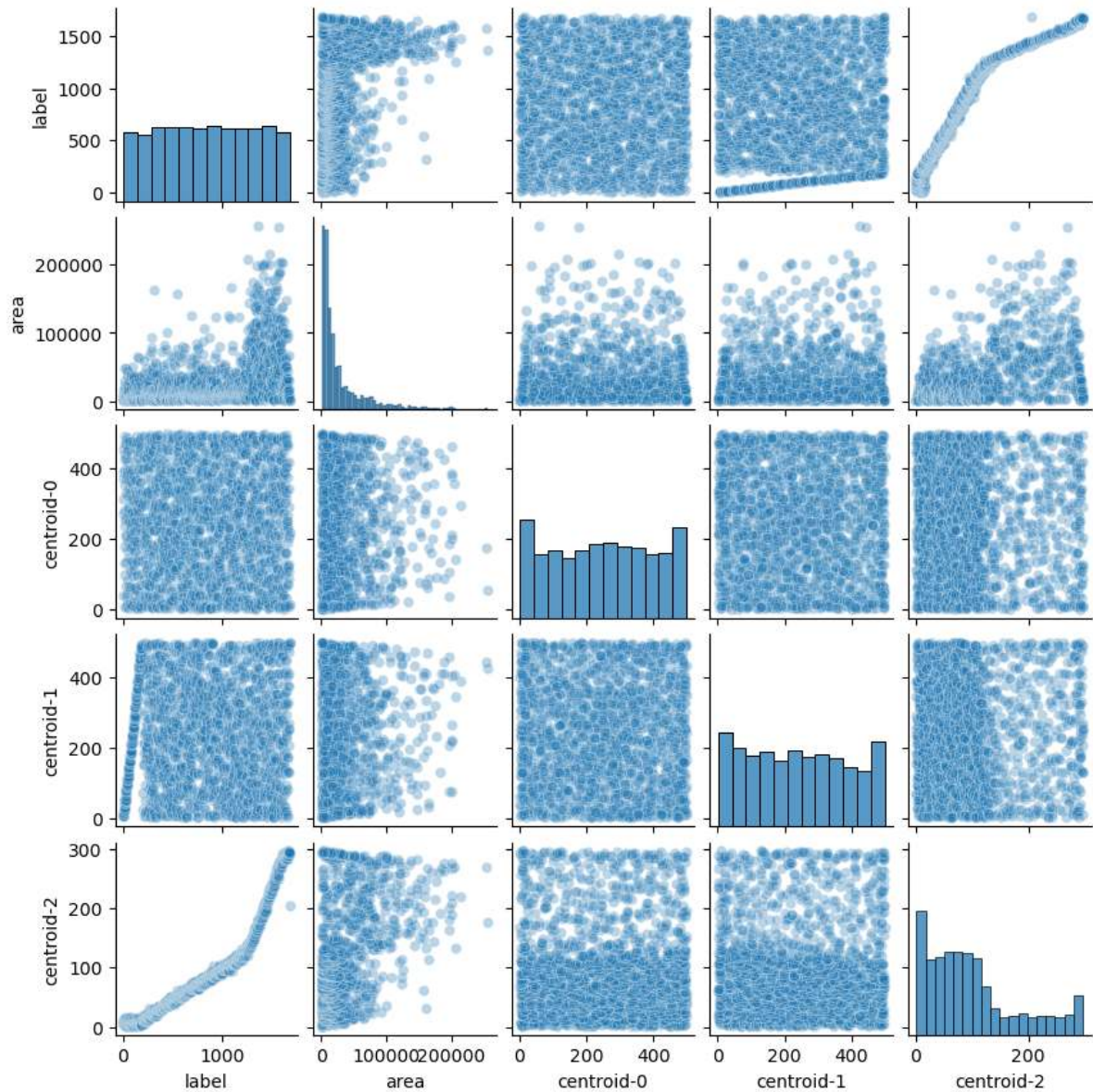


## Visualize the pruned table



## A pair plot of the properties

```
sns.pairplot(grains, vars=['label', 'area', 'centroid-0', 'centroid-1', 'centroid-2'],  
             height=1.75, plot_kws={'alpha': 0.3});
```

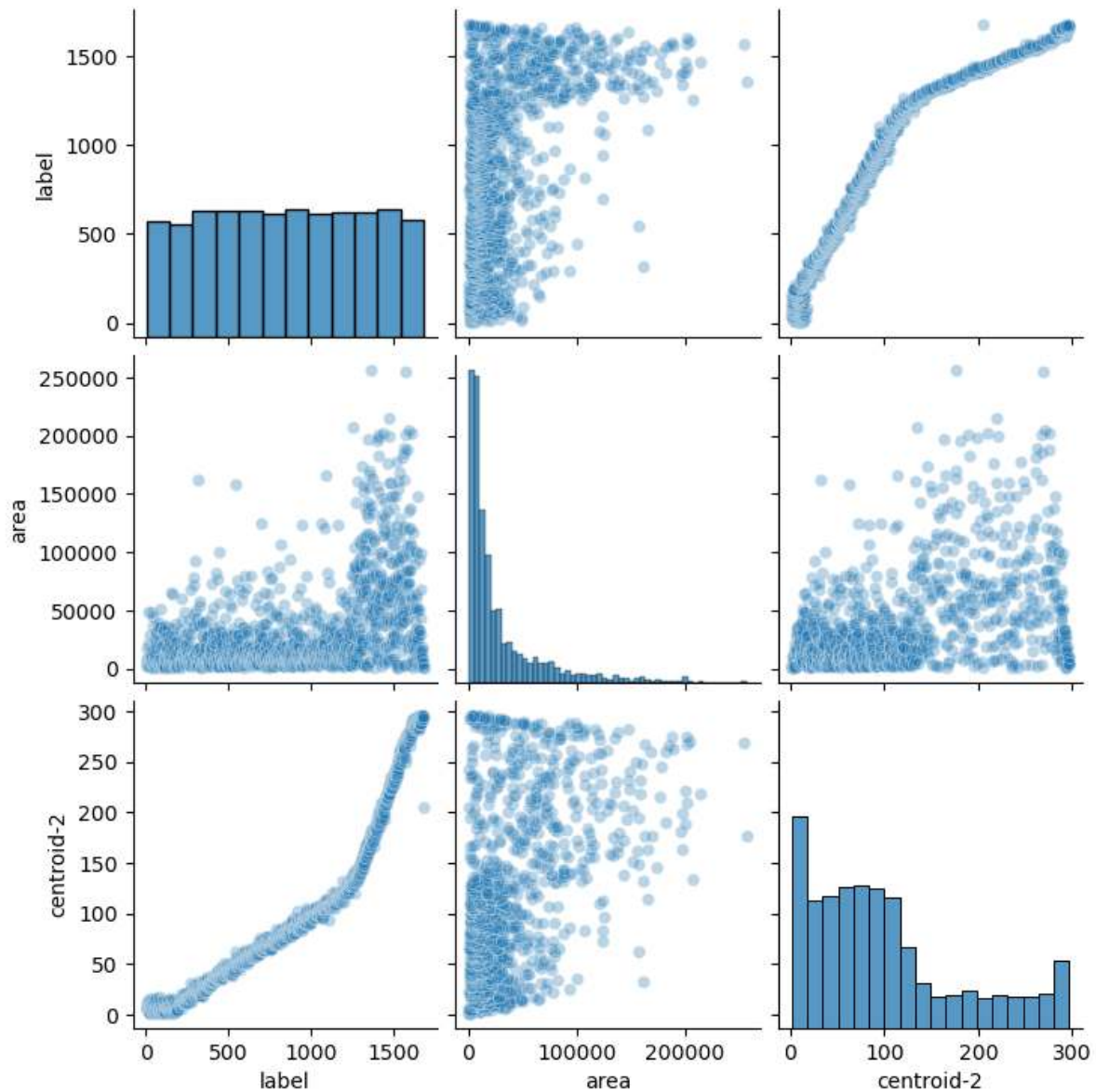


We can see clear property pairs that provide information to support the classification:

- The item `label` vs `area` or `centroid-2`.
- The item `area` vs `centroid-2`

## Too much information - A closer look at the relevant pairs

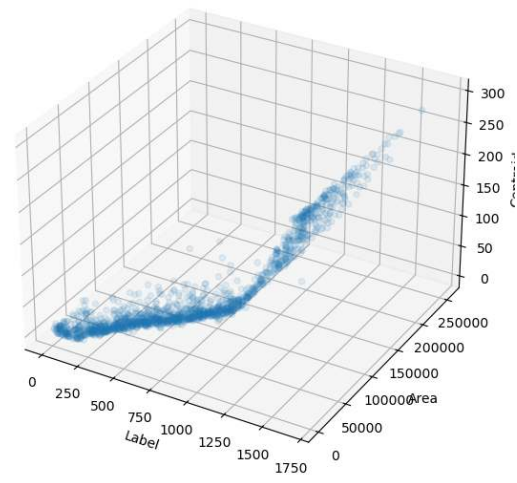
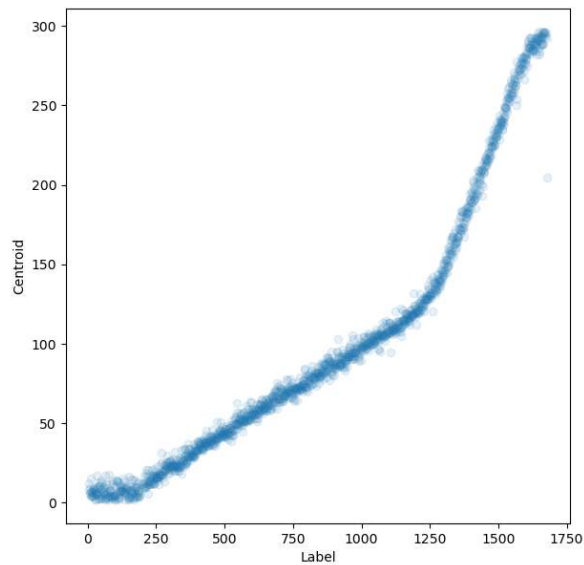
```
sns.pairplot(grains, vars=['label', 'area', 'centroid-2'], plot_kws={'alpha': 0.3});
```



We see here that there is a clear connection between the label and area and centroid. This does normally not hold true, but is rather a coincidence because of the way the labelling algorithm is implemented. It increases the label numbers with the linear scan over the coordinates. Still, in our case we can use this to our advantage as it clearly helps us to find the cut line between the two groups.



### Looking for clusters



These are logical pairings - the grains fractions are

- packed in the vertical direction
- the watershed labels run in the vertical direction.

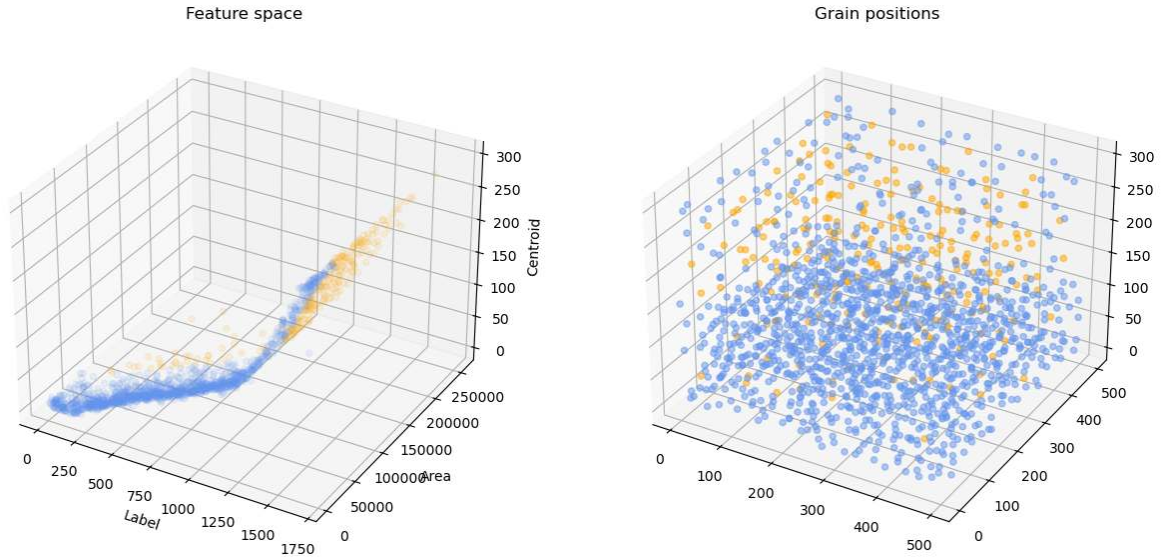
### K-Means to cluster the grains

```
from sklearn.cluster import KMeans

kmeans= KMeans(2)
x=kmeans.fit_predict(grains[['label', 'area', 'centroid-2']].values)
grains['group']=x.tolist()
grains.sample(5, random_state=30)
```

|      | label | area     | centroid-0 | centroid-1 | centroid-2 | fg   | small | group |
|------|-------|----------|------------|------------|------------|------|-------|-------|
| 852  | 853   | 6719.0   | 255.218634 | 494.421789 | 84.453788  | True | False | 0     |
| 444  | 445   | 100690.0 | 435.006982 | 60.617301  | 36.261237  | True | False | 1     |
| 538  | 539   | 4143.0   | 120.373401 | 395.345643 | 53.542119  | True | False | 0     |
| 1340 | 1341  | 155857.0 | 447.615680 | 232.692969 | 162.332625 | True | False | 1     |
| 1605 | 1606  | 27592.0  | 108.840533 | 205.322014 | 287.128298 | True | False | 0     |





This is not really what we wanted!

Looking closer at the ranges of each feature we see that the Area more than 1000-fold greater.

**We need to scale the features**

### Try k-means to cluster the grains - normalized features

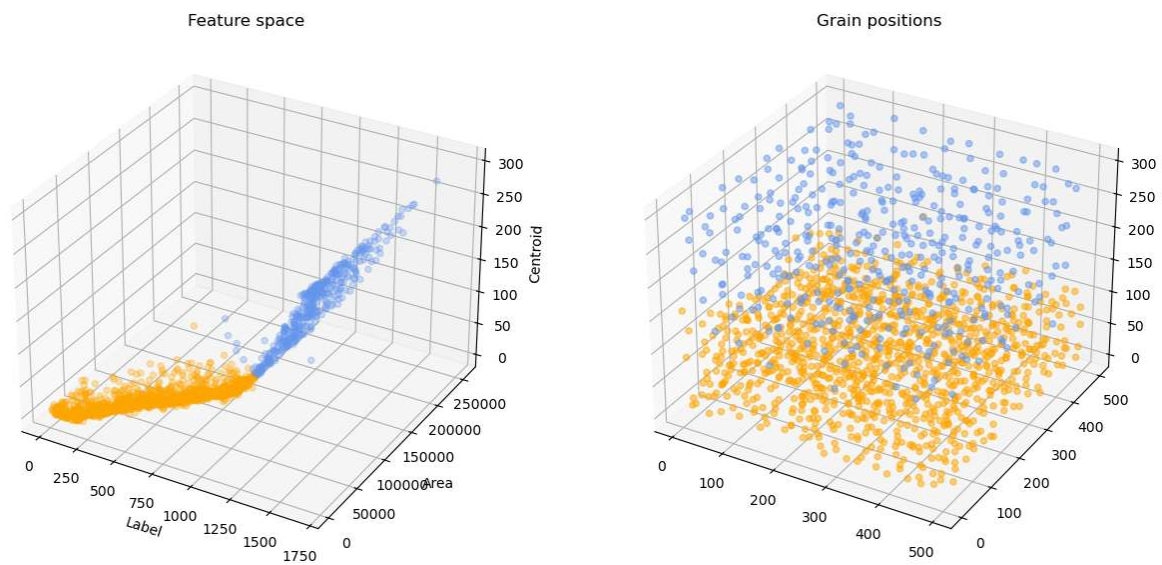
With each vector  $v$   $v_{normalized} = \frac{v - \text{mean}(v)}{\text{std}(v)}$

This kind of statistical normalization is very common in machine learning applications. The models are often biased towards the variable with the greatest variance.

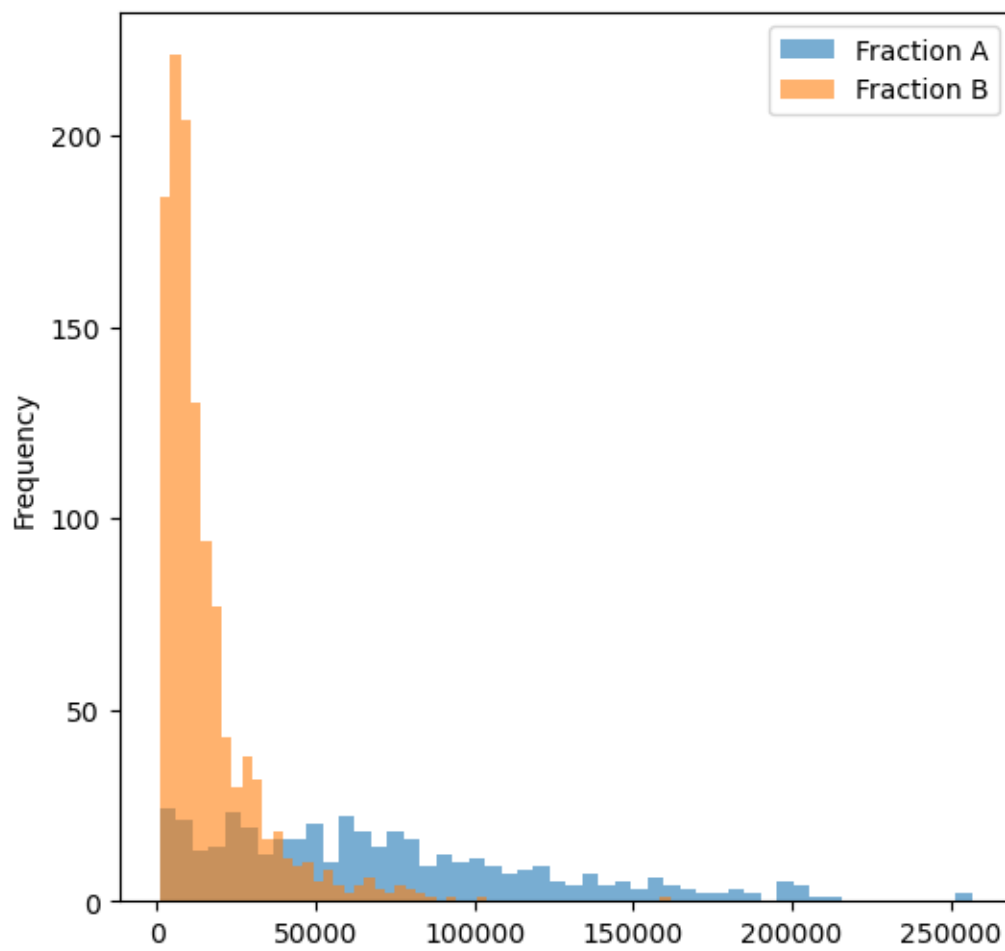
All variables have values in the same range after normalization and thus also equally treated.

```
grains['nlabel'] = (grains['label']-grains['label'].mean())/grains['label'].std()
grains['narea']  = (grains['area']-grains['area'].mean())/grains['area'].std()
grains['nz']     = (grains['centroid-2']-grains['centroid-2'].mean())/grains[
    ↪ 'centroid-2'].std()

kmeans= KMeans(2)
x=kmeans.fit_predict(grains[['nlabel','narea','nz']].values)
grains['group']=x
```

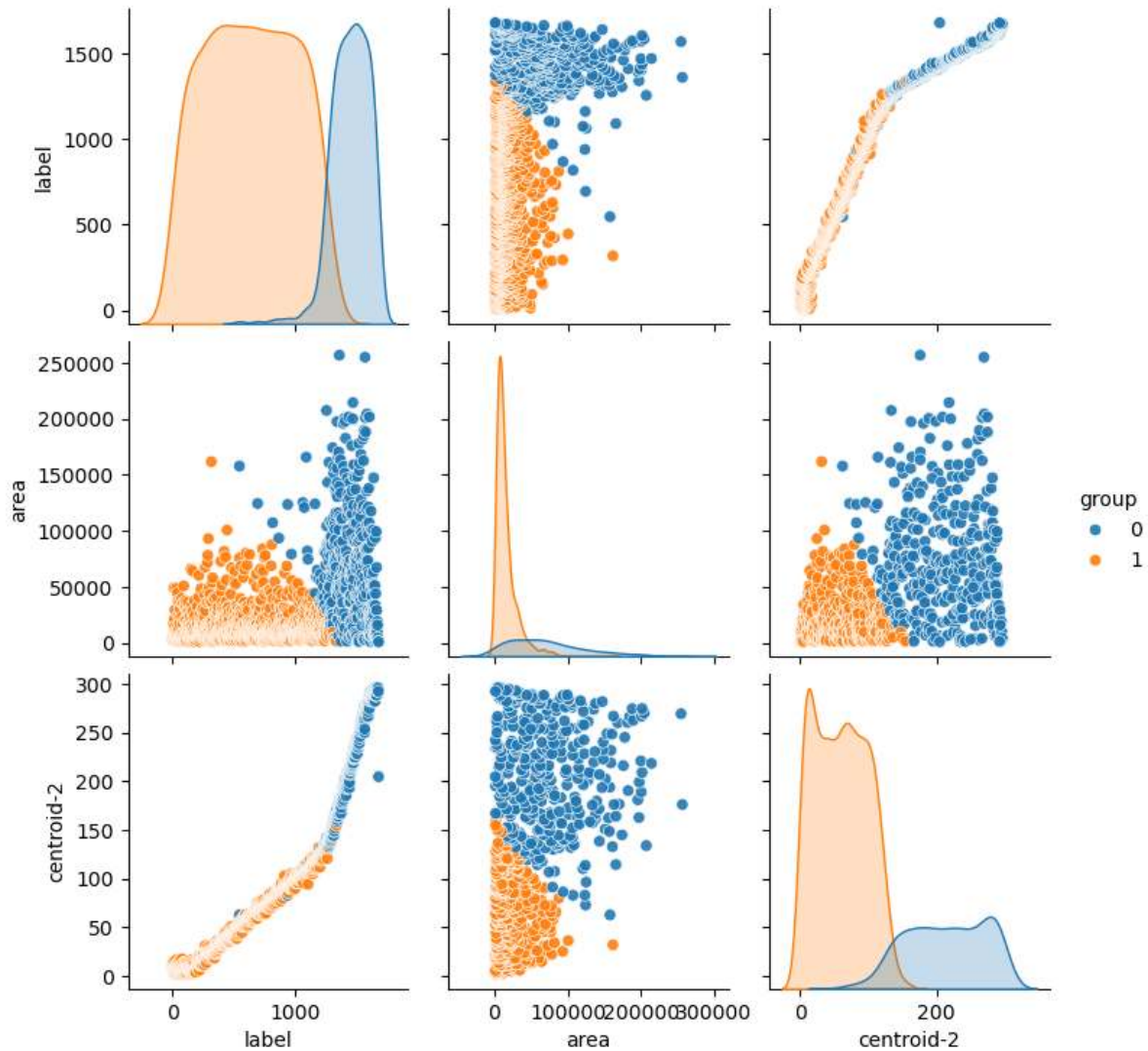


### Looking at the grains size histograms



## Return to the pair plot

```
sns.pairplot(grains, vars=['label', 'area', 'centroid-2'], hue='group', plot_kws={'alpha': 0.9});
```



## Assign classes back to the image

Finally, we come to the last step assigning group labels to the grains.

```
from skimage.morphology import reconstruction

def assignGroups(img, itemdf) :
    seeds = np.zeros_like(bi).astype('float')

    avg0 = itemdf[itemdf['group']==0]['centroid-2'].mean()
    avg1 = itemdf[itemdf['group']==1]['centroid-2'].mean()

    if avg1 < avg0 :
```

(continues on next page)

(continued from previous page)

```

itemdf['group']=1-itemdf['group']

for i,row in itemdf.iterrows() :
    seeds[int(np.floor(row['centroid-0'])), int(np.floor(row['centroid-1'])),
    int(np.floor(row['centroid-2']))]=2+row['group']

a=img*seeds.max()+1.0
seeds[a<seeds] = 0
glbl = reconstruction(seeds,a,method='dilation')-1

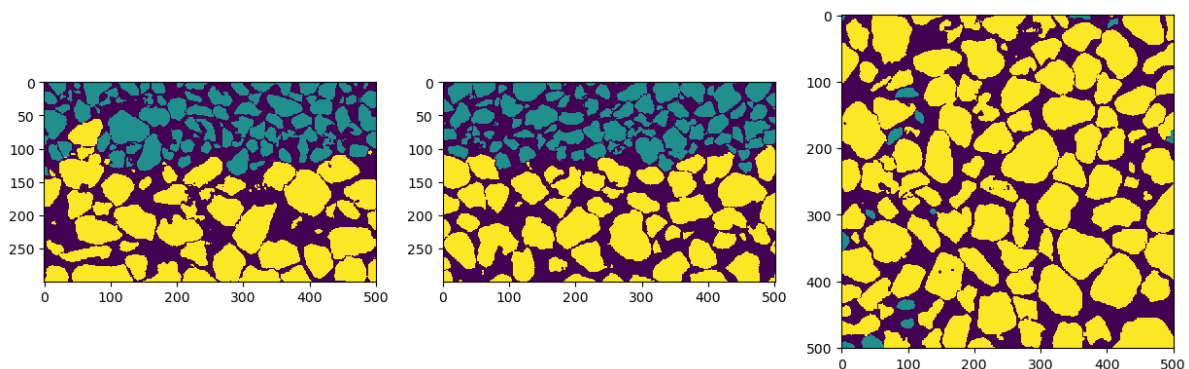
return glbl

```

You could also use the `skimage.segmentation.flood_fill` instead of the reconstruction algorithm. That would require you to implement a loop over the seed values.

### Apply the label assignment to the grains

```
glbl = assignGroups(bi,grains);
```



### The interface surface

The purpose of the analysis was to identify the surface between the small and the large grains in the image. So, the last step is to find this surface.

This sample was nicely aligned so that the separation is nearly horizontal. This condition allows for a simplified algorithm to identify the separation surface.

For each voxel in the xy-plane find:

- Small grain surface: the last z-position containing a small grain.
- Large grain surface: the first z-position containing a large grain.
- The middle surface

This can be done using the `argmax`-function.

```

import skimage.filters as flt

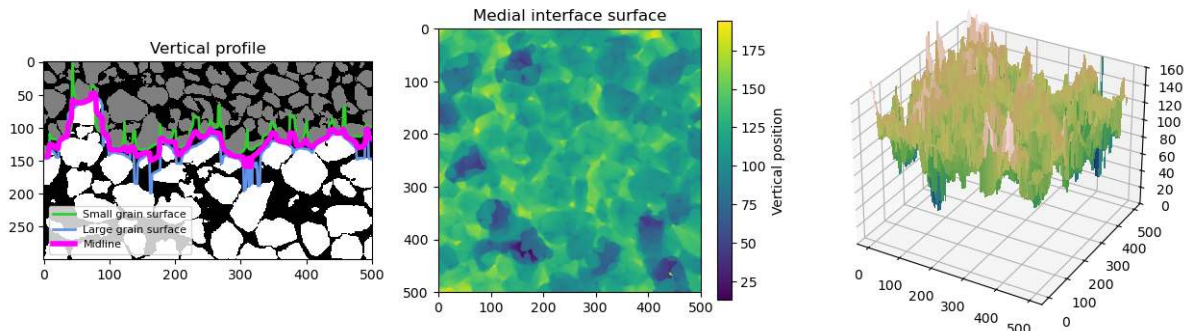
bottomB = glbl.shape[2]-np.argmax(glbl[:, :, :-1]==1,axis=2)
topA = np.argmax(glbl==2,axis=2)

```

(continues on next page)

(continued from previous page)

```
mAB = flt.median(0.5*(topA+bottomB), footprint=np.ones([7,7]))
```



## 0.15 Summary

### 0.15.1 Particle distributions

- Ways to measure the spatial distribution of particles in an image.

### 0.15.2 Skeletons

- Minimal structure of an object
- Object topology is preserved

### 0.15.3 Watershed segmentation

- Labels touching item
- May oversegment
- Frequently used algorithm

### 0.15.4 Putting things together

- A full workflow for grain analysis

## 0.16 Next time on QBI

- Uncertainties
- Statistical analysis

... but first - enjoy Easter break!

